



Module M244 • Veille Technologique

RADAR

Année universitaire 2025 – 2026

Remerciements

Nous adressons nos remerciements les plus sincères au Pr. Younes Wadii, enseignant du module M244 Veille Technologique à l'École Nationale des Sciences Appliquées de Tétouan, pour la rigueur de son enseignement et la clarté de la méthodologie qu'il nous a transmise. Sans le cadre conceptuel du cycle de veille, des grilles d'analyse SWOT et PESTEL, de la grille CRAAP et des signaux faibles que nous avons reçus en cours, ce projet n'aurait pu prendre la forme d'un produit opérationnel. Nous lui sommes également reconnaissants pour la liberté qu'il nous a laissée dans le choix d'un sujet ambitieux, et pour son ouverture d'esprit face à une réalisation qui croise sa spécialité avec l'intelligence artificielle générative.

Résumé

Ce rapport présente RADAR, un outil pensé pour les entreprises qui veulent savoir chaque semaine ce que font leurs concurrents sans y passer plusieurs heures chaque matin.

L'utilisation est directe. On indique le nom de son entreprise et celui de ses concurrents principaux, et RADAR s'occupe du reste. Chaque semaine, il parcourt le web pour repérer les nouvelles informations qui touchent ces concurrents : changements de prix, recrutements importants, lancements de produits, levées de fonds, campagnes de communication. L'intelligence artificielle qui pilote l'outil lit ces informations, fait le tri entre celles qui sont fiables et celles qui ne le sont pas, et ne retient que ce qui mérite vraiment l'attention.

Chaque lundi, un compte-rendu structuré est mis à disposition dans l'application, sur la page Rapports. On y trouve un résumé rapide des mouvements de la semaine, une analyse des forces et faiblesses de l'entreprise face à la concurrence, une lecture des grandes tendances de son secteur, une liste d'indices qui annoncent des évolutions à venir, et des recommandations classées par ordre d'importance. Le rapport se consulte directement dans le tableau de bord en ligne et s'exporte en PDF pour être téléchargé, archivé et partagé avec les membres de la direction lors des réunions ou comme livrable client.

Table des matières

Remerciements	1
Résumé	2
Liste des acronymes et abréviations	7
1 Introduction et vue d'ensemble	8
1.1 Introduction générale	8
1.2 Problématique et besoin identifié	8
1.3 Objectifs et solution	9
1.4 Méthodologie projet et planification	10
1.5 Cahier des charges simplifié	11
2 État de l'art	13
2.1 Concepts fondateurs de la veille stratégique	13
2.2 Panorama des outils de veille existants	13
2.3 Évolution des agents IA autonomes	14
2.4 Positionnement de RADAR	15
3 Architecture de RADAR	16
3.1 Schéma global du système	16
3.2 Choix d'architecture clés	17
3.3 Stack technique justifiée	18
3.4 Sécurité et isolation des sessions utilisateur	20
4 Implémentation du moteur agent	21
4.1 Infrastructure Docker	21
4.2 Configuration OpenClaw	21
4.3 Architecture skills versus agents	23
4.4 Les huit compétences SKILL.md	24
4.4.1 Deep Research	24
4.4.2 Orchestrateur	24
4.4.3 Collecteur	25
4.4.4 Évaluateur	26
4.4.5 Analyste PESTEL	26
4.4.6 Analyste SWOT	27
4.4.7 Détecteur de signaux faibles	27

4.4.8	Rédacteur	28
4.5	Intégration de DeepSeek V4 Pro	28
4.6	Intégration de Tavily et fallback DuckDuckGo	29
4.7	Communication entre agent et application web	29
4.8	Gestion des timeouts et fiabilité du pipeline	31
5	Implémentation de l'application web	32
5.1	Authentification et gestion des utilisateurs	32
5.2	Onboarding guidé en trois étapes	33
5.3	Garde d'accès et condition d'onboarding	38
5.4	Le tableau de bord	38
5.5	Modèle de données et persistance	43
6	Tests et validation du système	44
6.1	Tests unitaires	44
6.1.1	Cas Marjane : test isolé de Deep Research	44
6.1.2	Cycle complet : rapport-test-005	44
6.1.3	Échecs rencontrés et corrections	45
6.2	Tests d'intégration	46
7	Bilan et perspectives	48
7.1	Analyse critique	48
7.1.1	Forces du projet	48
7.1.2	Faiblesses et limites de la V1	49
7.1.3	Difficultés rencontrées et solutions trouvées	49
7.2	Apports du projet	50
7.3	Conclusion générale	51
7.4	Perspectives d'évolution	51
8	Bibliographie et références	53

Table des figures

1.1	Diagramme de Gantt de la planification du projet RADAR	11
3.1	Architecture globale de RADAR	16
4.1	Configuration retenue : un agent chargeant huit compétences	23
5.1	Page de connexion	33
5.2	Création de compte	33
5.3	Onboarding, étape 1 : profil de l'entreprise (nom et site web)	34
5.4	Onboarding, étape 2 : saisie des concurrents et suggestions automatiques	35
5.5	Onboarding, étape 3 : sélection des axes de surveillance	36
5.6	Page de confirmation de fin d'onboarding	36
5.7	Résultat de la recherche d'arrière-plan (Deep Research) sur l'entreprise	37
5.8	Résultats détaillés de la recherche sur l'entreprise présentés à la fin de l'onboarding	37
5.9	Page Brief : lancement de la veille depuis le tableau de bord	39
5.10	Page Brief : veille terminée	39
5.11	Page Signaux faibles	40
5.12	Page Concurrents	40
5.13	Page SWOT (dernière veille en date)	40
5.14	Page PESTEL	41
5.15	Page Rapports	41
5.16	Export PDF d'un rapport de veille	42
5.17	Page Cycles de veille	42
5.18	Page Paramètres	43
5.19	Paramètres : profil de l'entreprise connectée	43

Liste des tableaux

1.1	Outils de travail et de coordination utilisés sur le projet RADAR	10
1.2	Cahier des charges synthétique de RADAR V1	12
2.1	Panorama comparé des principaux outils de veille concurrentielle	14
3.1	Stack technique de RADAR et justification de chaque brique	19
4.1	Comparatif des modèles LLM évalués pour RADAR	28
4.2	Routes internes consommées par le moteur agent	30
6.1	Résultats du cycle de test rapport-test-005 (mai 2026)	45
6.2	Métriques du premier test d'intégration de bout en bout (compte inwi, 30 mai 2026)	46
7.1	Principales difficultés techniques et solutions apportées	50

Liste des acronymes et abréviations

API	Application Programming Interface
BDIA	Big Data et Intelligence Artificielle
CRAAP	Currency, Relevance, Authority, Accuracy, Purpose
CRUD	Create, Read, Update, Delete
ENSA	École Nationale des Sciences Appliquées
ERD	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
ICP	Ideal Customer Profile
JSON	JavaScript Object Notation
LLM	Large Language Model
M244	Code du module Veille Technologique
MVP	Minimum Viable Product
ORM	Object Relational Mapping
OAuth	Open Authorization
PDF	Portable Document Format
PESTEL	Politique, Économique, Sociologique, Technologique, Environnemental, Légal
PME	Petite et Moyenne Entreprise
PRD	Product Requirements Document
RGPD	Règlement Général sur la Protection des Données
SaaS	Software as a Service
SSO	Single Sign-On
SWOT	Strengths, Weaknesses, Opportunities, Threats
TPM	Tokens Per Minute
UI	User Interface

Chapitre 1

Introduction et vue d'ensemble

1.1 Introduction générale

Ce rapport rend compte du travail mené pour concevoir et développer RADAR, dans le cadre du module M244 Veille Technologique du cycle ingénieur Big Data et Intelligence Artificielle de l'École Nationale des Sciences Appliquées de Tétouan.

Le module M244 forme les futurs ingénieurs aux pratiques de la veille stratégique : cerner un besoin d'information, collecter, évaluer les sources, analyser avec les grilles SWOT et PESTEL, repérer les signaux faibles, puis diffuser des résultats exploitables. Pour le valider, il faut un projet de fin de semestre qui prouve qu'on maîtrise ces notions sur un cas concret.

Plutôt que de mener une veille sur une seule entreprise, nous avons choisi de livrer un logiciel : une plateforme qui automatise tout le cycle de veille, pour n'importe quelle entreprise, en s'appuyant sur les grands modèles de langage récents. Trois raisons à ce choix. D'abord, transformer chaque notion du cours en une brique technique qu'on peut réellement tester. Ensuite, aboutir à un outil utilisable sur le terrain, y compris par un dirigeant de PME ou un consultant sans bagage technique. Enfin, repartir du semestre avec une réalisation concrète à présenter en entretien, que ce soit en stratégie, en data ou en IA appliquée.

1.2 Problématique et besoin identifié

Toute entreprise qui évolue dans un marché concurrentiel doit savoir ce que font ses rivaux. Les dirigeants de PME, les responsables stratégie et les consultants le savent bien : un concurrent qui ajuste ses prix, recrute en masse, lève des fonds ou refond son offre, c'est une information qui peut faire basculer une décision. Le problème n'est pas que ces informations manquent, au contraire, elles sont largement publiques. Le problème, c'est que dans une organisation de taille moyenne, personne n'a le temps de les collecter chaque semaine, de les recouper, de les évaluer puis de les mettre en forme.

En pratique, la veille concurrentielle prend rarement une forme aboutie dans les PME et même dans les grandes entreprises. La situation la plus fréquente : aucune routine formalisée, la direction se contente du bouche-à-oreille sectoriel et de ce qui remonte sur LinkedIn. Variante un peu plus mature : un collaborateur dédie une heure

ou deux chaque matin à naviguer sur les sites concurrents, scroller les réseaux, lire la presse économique, et consigne ce qu'il en retient dans un tableur sans grille d'analyse. Cas plus rare : l'entreprise s'abonne à Similarweb, SEMrush ou Crayon, mais finit par constater que ces outils répondent surtout à des besoins SEO et marketing, pas à une veille stratégique au sens du cycle enseigné en M244.

Les mêmes faiblesses reviennent dans ce mode de fonctionnement. La découverte est tardive : un changement de prix ou une levée de fonds remonte par la presse des semaines après les faits. Les sources ne sont pas évaluées, si bien que rumeurs et informations confirmées se retrouvent sur le même plan. Rien n'est mis en forme en SWOT ou en PESTEL, ce qui complique tout raisonnement sur les axes stratégiques. Les signaux faibles, ces indices précoces et dispersés qui annoncent un mouvement de fond, passent inaperçus. Et il n'existe aucune mémoire centralisée : chaque revue repart de zéro.

Le besoin est simple à formuler : disposer chaque semaine d'une synthèse concurrentielle prête à lire, fondée sur la méthodologie M244, sans qu'un humain ait à la produire à la main. L'outil doit couvrir le cycle complet, de la collecte à la diffusion en passant par l'analyse, et ne remonter que des informations vérifiées et recoupées. C'est précisément ce que fait RADAR.

1.3 Objectifs et solution

L'objectif principal est simple : une plateforme qui déroule seule le cycle de veille, de la collecte jusqu'à la livraison du rapport hebdomadaire. Cinq objectifs opérationnels en découlent.

Automatiser la collecte web sans se faire bloquer par les protections anti-bot, en passant par un fournisseur de recherche pensé pour les agents. Évaluer chaque source avec la grille CRAAP avant de l'utiliser, pour que la fiabilité de ce qui remonte reste traçable. Produire des SWOT et des PESTEL qui tiennent la route face à un examen critique. Détecter les signaux faibles, ces indices précoces d'une évolution du secteur. Et livrer le tout dans un tableau de bord sobre, où chaque rapport se consulte, se télécharge en PDF et se partage.

La solution repose sur un agent d'intelligence artificielle piloté par OpenClaw, un runtime open source qui exécute des compétences décrites au format Markdown. L'agent RADAR en mobilise huit. Deep Research dresse le profil de l'entreprise utilisatrice à l'inscription, en dehors du cycle hebdomadaire. L'orchestrateur coordonne le reste. Le collecteur interroge Tavily pour ramener des sources fraîches sur chaque concurrent ; l'évaluateur leur attribue un score CRAAP ; deux analystes produisent la PESTEL puis la SWOT ; un détecteur cherche les signaux faibles ; et un rédacteur met tout en forme dans un rapport narratif. L'orchestrateur enchaîne ces compétences

avec le mécanisme de sous-sessions d'OpenClaw, ce qui isole les contextes et garde la consommation de tokens sous contrôle.

Côté utilisateur (dirigeant de PME ou de grand groupe, consultant, étudiant en stratégie), tout passe par une application web sous Next.js 16. À l'inscription, il donne le nom de son entreprise et son site, ajoute ses concurrents prioritaires et choisit ses axes de surveillance. Le moteur agent prend ensuite le relais : chaque lundi à six heures, un cron lance l'orchestrateur, qui déroule toute la chaîne, écrit les résultats en base via les routes internes et met le rapport à disposition. La consultation se fait dans un tableau de bord : feed des alertes, grilles SWOT et PESTEL, timeline des mouvements détectés, signaux faibles. Sur la page Rapports, l'utilisateur ouvre le rapport, le télécharge en PDF et le partage en comité de direction ou comme livrable client.

1.4 Méthodologie projet et planification

Nous avons travaillé en cycles courts, d'une à deux semaines, dans l'esprit du développement itératif et du Lean Startup. Plutôt que de tout spécifier avant d'écrire la première ligne de code, chaque cycle se terminait par quelque chose de montrable : un composant assemblé ou une série de tests d'intégration. C'est ce qui nous a permis d'encaisser sans trop de casse les décisions qu'on ne pouvait pas voir venir, comme l'abandon de GPT-4o pour DeepSeek V4 Pro (limites de tokens par minute) ou le passage de DuckDuckGo à Tavily (blocage des rafales de requêtes).

Les outils mobilisés pour la coordination et le développement sont résumés dans le tableau 1.1.

Table 1.1. Outils de travail et de coordination utilisés sur le projet RADAR

Catégorie	Outils retenus
Versionnement et collaboration	Git, GitHub (branches <code>feature/agent</code> et <code>feature/web</code> , pull requests)
Gestion du monorepo	pnpm 10, Turborepo
Documentation interne	Fichiers Markdown versionnés
Conteneurisation	Docker, Docker Compose, image officielle <code>ghcr.io/openclaw/openclaw:latest</code>
Tests d'intégration	Mock API HTTP en Node.js qui simule les routes Next.js côté agent
Éditeur de code	Visual Studio Code, LaTeX (pour le rapport), ESLint, Prisma
Communication équipe	WhatsApp pour le quotidien, sessions de travail synchrones le week-end

La planification du projet s’est étalée entre mars et juin 2026, en suivant l’enchaînement de phases présenté dans la figure 1.1.

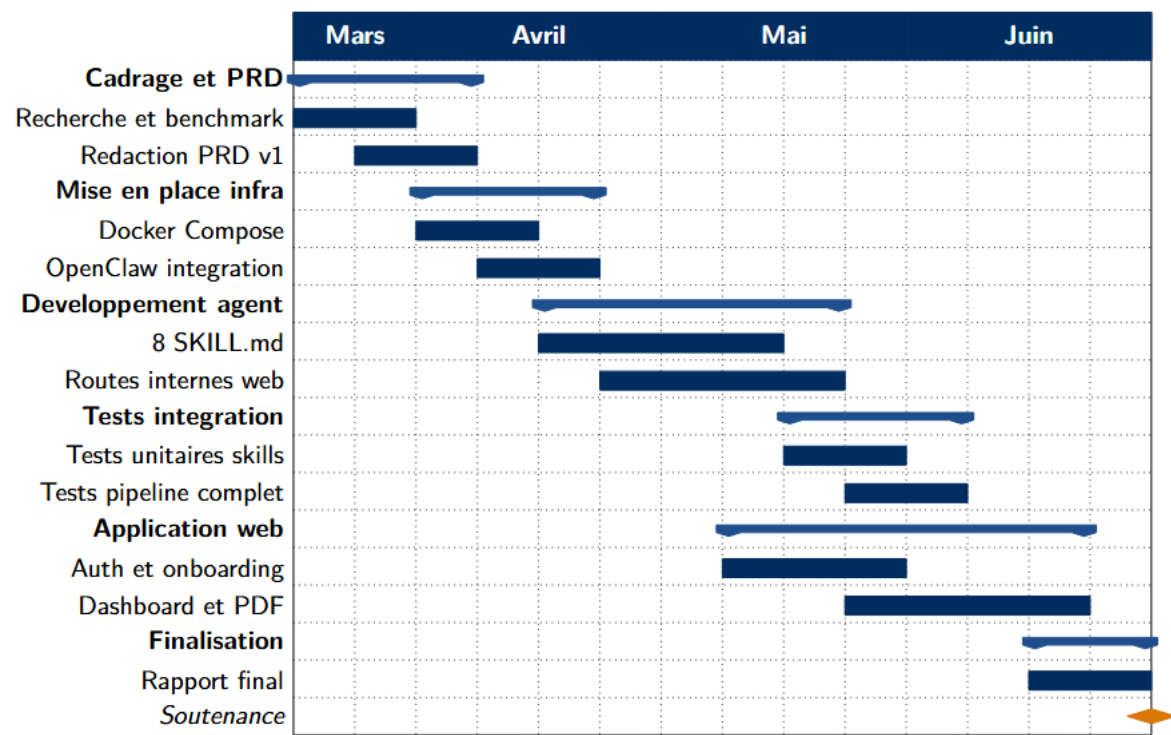


Figure 1.1. Planification du projet RADAR sur quatre mois (mars a juin 2026)

Figure 1.1. Diagramme de Gantt de la planification du projet RADAR

1.5 Cahier des charges simplifié

RADAR n’est pas qu’un exercice scolaire : nous l’avons pensé comme une offre qu’on pourrait facturer à un client. Le tableau 1.2 reprend le cahier des charges de la version 1 dans cette logique.

Table 1.2. Cahier des charges synthétique de RADAR V1

Rubrique	Engagement
Besoin client	Disposer chaque semaine d'un rapport de veille concurrentielle structuré, sans intervention manuelle, couvrant entre un et dix concurrents nommés par l'utilisateur.
Fonctionnalités clés	Inscription guidée avec enrichissement automatique du profil entreprise (Deep Research), cycle de veille hebdomadaire déclenché par cron, dashboard temps réel, consultation et export PDF des rapports.
Contraintes techniques	Hébergement Docker mutualisé ou self-hosted (VPS Hetzner CX21), dépendance externe à DeepSeek (modèle LLM) et Tavily (recherche web), réseau interne entre les services, persistance PostgreSQL.
Indicateurs de succès	Rapport produit chaque semaine, le lundi avant 7h00, sources évaluées CRAAP avec score moyen supérieur à 14 sur 20, recoupement minimum à deux sources indépendantes pour chaque alerte, taux d'échec inférieur à 5 pour cent par cycle.
Délai de livraison	Une version opérationnelle déployée sur VPS au plus tard une semaine avant la soutenance, c'est-à-dire mi-juin 2026.
Ressources mobilisées	Deux développeurs étudiants en cycle ingénieur, environ vingt heures hebdomadaires chacun sur la durée du semestre.
Résultats attendus	Une plateforme utilisable en mode démonstration pour le jury, un rapport documenté, un dépôt Git public, et la capacité à soutenir une démonstration en direct sur un compte test.

Ce cahier des charges servira de référence tout au long du rapport : chaque choix technique présenté dans les chapitres suivants pourra être confronté aux engagements pris ici, et le chapitre 7 établira un bilan honnête de ce qui a été tenu et de ce qui reste à finaliser.

Chapitre 2

État de l'art

Ce chapitre replace RADAR dans deux contextes qui le précèdent : la veille stratégique comme discipline, et les agents IA comme technologie. Il reste court : l'idée est surtout de situer notre solution et de la distinguer de ce qui existe déjà sur le marché.

2.1 Concepts fondateurs de la veille stratégique

La veille stratégique a été formalisée dans la littérature francophone par Humbert Lesca. Le module M244 du Pr. Younes Wadiai l'enseigne à travers quatre concepts, que nous avons repris tels quels dans le moteur RADAR.

Le **cycle de veille** en cinq étapes (identification des besoins, collecte, analyse et traitement, diffusion et exploitation, mise à jour continue) sert de colonne vertébrale au pipeline : chaque compétence du moteur couvre un segment du cycle.

Pour évaluer les sources, nous utilisons la **grille CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose), proposée par Sarah Blakeslee en 2004. Chaque critère vaut quatre points, soit un score sur vingt. L'Évaluateur le calcule pour chaque source, et un seuil de cinq sur vingt déclenche le filtrage.

L'analyse stratégique mobilise les grilles **SWOT** (forces et faiblesses internes, opportunités et menaces externes) et **PESTEL** (six dimensions : politique, économique, sociologique, technologique, environnementale, légale). RADAR produit la grille SWOT comme la grille PESTEL au rythme hebdomadaire.

Enfin, RADAR détecte les **signaux faibles** théorisés par Igor Ansoff en 1975 dans *Managing Strategic Surprise by Response to Weak Signals*. La compétence dédiée croise les sources mineures (forums, blogs, offres d'emploi) et majeures sur une fenêtre glissante de trente jours stockée en base PostgreSQL, pour faire émerger des récurrences qui annoncent un mouvement de fond.

2.2 Panorama des outils de veille existants

Le marché de la veille concurrentielle est dense mais éclaté. Les outils les plus connus répondent en fait à des besoins très différents, et la plupart ne couvrent pas la veille stratégique méthodologique au sens du M244. Le tableau 2.1 récapitule les solutions que nous avons regardées au moment du cadrage.

Table 2.1. Panorama comparé des principaux outils de veille concurrentielle

Outil	Cœur métier	Limite par rapport à RADAR
Similarweb	Mesure du trafic web, classement par catégorie, sources d'audience	Aucune analyse SWOT ni PESTEL, pas de détection de signaux faibles, orienté SEO et marketing
SEMrush	Positionnement de mots-clés, backlinks, suivi de la présence digitale	Même limite : centré sur le SEO, ne couvre pas la veille stratégique
Crayon	Veille concurrentielle entreprise avec battlecards et alerting	Très orienté sales enablement grand compte, prix prohibitif pour une PME
Mention	Social listening, mentions de marque, sentiment	Pas de cadre méthodologique, ne produit pas de grille d'analyse
Brandwatch	Social listening grand compte avec analytics avancés	Même remarque, plus la barrière du prix
Google Alerts	Notification de nouveautés à partir de mots-clés	Beaucoup trop bruyant, aucune évaluation des sources, pas de structuration
Feedly	Agrégateur de flux RSS avec couche IA optionnelle	Centré sur la lecture, ne génère pas de synthèse stratégique

Aucun ne prend le cycle M244 comme fil conducteur. C'est là que RADAR se place : produire chaque semaine des livrables stratégiques structurés par une méthode explicite, pour des PME et des consultants indépendants qui n'ont ni le budget ni le besoin des suites entreprise.

2.3 Évolution des agents IA autonomes

En intelligence artificielle, le terme *agent* désigne depuis longtemps un programme capable de percevoir son environnement et d'agir pour atteindre un objectif. Tout a changé en 2023 avec les grands modèles de langage : le moteur de raisonnement n'est plus une suite de règles écrites à la main, mais un LLM auquel on donne accès à des outils externes et à une mémoire de travail. AutoGPT, publié en avril 2023, a popularisé cette approche, suivi par des frameworks plus aboutis comme LangChain et son évolution LangGraph.

Les architectures actuelles reposent sur quatre briques : un LLM comme moteur de raisonnement, des outils pour agir au-delà du texte (recherche web, lecture de fichiers, exécution de code), une mémoire à court et long terme, et une boucle de contrôle qui

décide quand mobiliser chaque outil.

OpenClaw, le runtime que nous avons retenu pour RADAR, intègre ces briques dans un seul binaire compatible avec l'API OpenAI Chat Completions. Les comportements d'un agent se décrivent dans des fichiers Markdown `SKILL.md`, lisibles et modifiables sans recompilation. La fonction `sessions_spawn` crée des sous-sessions isolées indispensables à un pipeline séquentiel, le cron natif gère le déclenchement hebdomadaire, et les outils `web_search`, `web_fetch` et `browser` sont disponibles sans configuration. Face à un paysage mouvant (Claude Agent SDK, Mastra, LangGraph, plateformes hébergées), notre choix repose sur le caractère open source du code, la compatibilité native avec l'API OpenAI qui ouvre la porte à tous les modèles disponibles dans cette interface, et la lisibilité immédiate du format `SKILL.md`.

2.4 Positionnement de RADAR

RADAR ne cherche pas à remplacer Similarweb, ni à faire l'agrégateur de flux, le social listener ou le framework d'agents généraliste. Son terrain est plus étroit : automatiser le cycle de veille M244 pour des PME et des consultants, en transformant chaque notion du cours en une fonctionnalité visible dans le produit.

Cette spécialisation est à la fois la force et la limite du projet. Force, parce qu'elle assure une cohérence méthodologique que les outils généralistes n'ont pas : chaque source collectée est évaluée par CRAAP, chaque alerte est recoupée, chaque rapport suit la structure attendue par le cours. Limite, parce qu'elle restreint le périmètre fonctionnel et nous oblige à laisser de côté des sujets adjacents comme la veille brevet, la veille réglementaire ou la surveillance de la réputation en ligne. Ces sujets pourraient faire l'objet d'extensions, dont nous discutons dans les perspectives du chapitre 7.

Chapitre 3

Architecture de RADAR

Ce chapitre décrit l'architecture technique de RADAR : comment les composants s'articulent, quelles décisions ont structuré la conception, quelle stack nous avons retenue et comment nous isolons les sessions utilisateur.

3.1 Schéma global du système

RADAR fonctionne sur trois services Docker indépendants qui collaborent via un réseau interne. Le premier est l'application web bâtie sur Next.js qui sert d'interface utilisateur et de couche de persistance. Le deuxième est le moteur agent OpenClaw qui exécute les huit compétences spécialisées. Le troisième est la base de données PostgreSQL qui stocke profils, sources, analyses et rapports. La figure 3.1 schématise cette topologie.

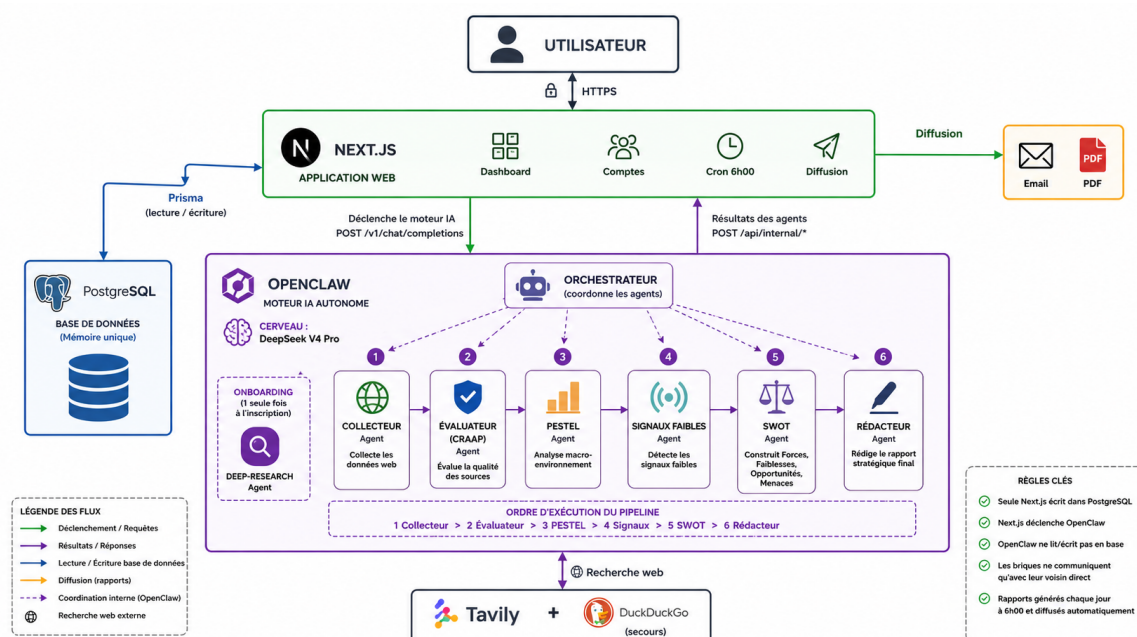


Figure 3.1. Architecture globale de RADAR

Le flux nominal d'un cycle de veille suit toujours la même séquence. Un cron déclenche l'orchestrateur chaque lundi à six heures du matin, ou un bouton dans le dashboard utilisateur peut le déclencher à la demande. L'orchestrateur lit le profil utilisateur passé en paramètre du message initial, puis enchaîne les sous-sessions via

`sessions_spawn`. Chaque sous-session exécute une compétence : collecte, évaluation, analyses, détection des signaux faibles, rédaction. Après chaque étape, la compétence envoie ses résultats à l'application web via une route interne dédiée, et retourne son livrable à l'orchestrateur pour alimenter l'étape suivante. Une fois le cycle terminé, le rapport final est marqué comme achevé dans la base et devient consultable par l'utilisateur dans l'application, sur la page Rapports.

3.2 Choix d'architecture clés

Cinq décisions structurantes ont orienté la conception. Nous les listons ici parce qu'elles conditionnent tous les chapitres suivants.

Décision 1 : un seul agent OpenClaw avec huit compétences, et non huit agents distincts

OpenClaw permet de définir plusieurs agents dans un même processus. Chaque agent dispose alors de son propre modèle, de ses propres outils, de sa propre mémoire. Cette approche multi-agent a un coût : la communication entre agents passe par des messages externes, ce qui complique l'orchestration et augmente la latence. Nous avons préféré une architecture simple : un seul agent (l'agent `main`) qui charge huit compétences au format SKILL.md. L'orchestration interne se fait alors via `sessions_spawn`, qui crée des sous-sessions isolées du même agent. Cette décision a été validée après lecture de la documentation OpenClaw : `sessions_spawn` ne sait pas cibler un autre agent, il opère sur l'agent courant.

Décision 2 : communication agent vers web via HTTP interne, sans signature HMAC

Les architectures de webhook classiques exigent une signature HMAC pour authentifier l'expéditeur. RADAR utilise un réseau Docker interne fermé : seuls les conteneurs déclarés dans le même `docker-compose.yml` peuvent atteindre les routes `/api/internal/*`. Cette isolation réseau rend la signature HMAC superflue en V1. Si le produit devait passer en multi-tenant hébergé sans isolation réseau, l'ajout d'une couche HMAC serait la première chose à faire ; nous l'avons documenté comme dette technique connue.

Décision 3 : modèle LLM DeepSeek V4 Pro, et non GPT-4o ni Claude Opus

Notre premier prototype utilisait GPT-4o. Il atteignait rapidement la limite de trente mille tokens par minute du Tier 1 OpenAI, ce qui bloquait la compétence Deep Research qui consomme à elle seule près de trente mille tokens sur le dernier appel. DeepSeek V4 Pro offre un contexte d'un million de tokens, une limite de débit élastique et un coût dix-sept fois inférieur à celui de GPT-4o, tout en restant compatible avec l'API OpenAI Chat Completions qu'OpenClaw exploite. La dégradation de qualité par rapport à Claude Opus 4.7 sur les analyses SWOT et PESTEL est restée acceptable lors des tests d'intégration.

Décision 4 : Tavily comme provider primaire de recherche web, DuckDuckGo en fallback

DuckDuckGo est gratuit et fonctionne sans clé API, ce qui en a fait notre premier choix. Mais le moteur applique une détection de bot agressive sur les rafales de requêtes, et la compétence Collecteur en émet typiquement quinze à vingt par cycle. Tavily, conçu pour les agents IA, retourne le contenu complet des pages (pas seulement les URL), n'applique pas de détection de rafales sur le free tier et offre mille requêtes par mois. La compétence Deep Research peut continuer à utiliser DuckDuckGo car ses requêtes sont espacées dans le temps, mais le Collecteur passe par Tavily.

Décision 5 : cron OpenClaw natif et orchestrateur sans persistance propre

OpenClaw embarque un scheduler cron natif qui déclenche un message initial vers un agent à l'heure configurée. Plutôt que d'ajouter une couche de scheduling externe comme pg-boss ou node-cron du côté Next.js, nous nous sommes contentés du cron OpenClaw. L'orchestrateur lui-même ne maintient pas d'état persistant : chaque cycle est indépendant et l'intégralité de la traçabilité remonte dans la base PostgreSQL via les routes internes. Cette simplicité réduit le nombre de composants à opérer en production.

3.3 Stack technique justifiée

Le tableau 3.1 résume la stack technique retenue et les raisons de chaque choix.

Table 3.1. Stack technique de RADAR et justification de chaque brique

Couche	Technologie	Raison du choix
Frontend	Next.js 16, React 19, Tailwind CSS 4	Standard de l'écosystème JavaScript, support natif du SSR et des Server Actions, communauté large
Authentification	Better-Auth	Support Next.js App Router, OAuth Google intégré, sessions DB-backed
Base de données	PostgreSQL 17	Robustesse, support JSONB pour les structures semi-structurées des analyses, gratuit
ORM	Prisma 6	Génération de types TypeScript à partir du schéma, migrations versionnées
Runtime Node	Node 24	Dernière LTS au moment du projet, performances améliorées
Monorepo	pnpm 10, Turborepo	Gestion partagée des dépendances, cache de build entre packages
Moteur agent	OpenClaw (image ghcr.io/openclaw/openclaw:latest)	Open source, format SKILL.md compatible, sessions_spawn natif
Modèle LLM	DeepSeek V4 Pro (deepseek/deepseek-v4-pro)	Contexte 1M tokens, coût dix-sept fois inférieur à GPT-4o, compatible API OpenAI
Recherche web	Tavily (primaire), DuckDuckGo (fallback)	Tavily conçu pour agents IA, pas de bot detection, contenu complet retourné
Conteneurisation	Docker, Docker Compose	Standard d'industrie, environnement reproductible local et production
Hébergement V1	Hetzner Cloud CX21	Coût maîtrisé (4 EUR par mois), serveurs allemands conformes RGPD

Nous avons choisi cette stack en visant deux choses à la fois : un coût d'exploitation minimal, parce que le projet n'a aucun budget commercial, et la maintenabilité, parce qu'un développeur extérieur doit pouvoir reprendre le code. Chaque brique est éprouvée et largement utilisée.

3.4 Sécurité et isolation des sessions utilisateur

La sécurité d'une plateforme qui agrège des données concurrentielles pour plusieurs clients mériterait un chapitre à elle seule. RADAR reste un MVP, mais nous avons quand même pris plusieurs précautions dès la conception, pour ne pas accumuler une dette de sécurité difficile à rattraper ensuite.

Isolation réseau. Les trois services Docker (Next.js, OpenClaw, PostgreSQL) communiquent uniquement sur un réseau privé déclaré dans le `docker-compose.yml`. Seuls les ports HTTPS de l'application web sont exposés vers l'extérieur. OpenClaw n'est jamais accessible depuis Internet, ce qui ferme la plus grande surface d'attaque potentielle.

Isolation des contextes utilisateur. Chaque cycle de veille déclenche une session OpenClaw dédiée, dont le contexte se limite au profil utilisateur transmis dans le message initial. Le mécanisme `sessions_spawn` crée des sous-sessions isolées pour chaque compétence du pipeline, ce qui empêche la fuite de données entre étapes. Aucune mémoire partagée entre utilisateurs n'est conservée côté agent ; toute la persistance passe par la base PostgreSQL où les enregistrements sont strictement rattachés à leur `userId`.

Secrets et clés API. Toutes les clés externes (`DEEPSEEK_API_KEY`, `TAVILY_API_KEY`, `OPENCLAW_INTERNAL_SECRET`) sont passées aux conteneurs via des variables d'environnement, jamais commitées dans le dépôt Git. Le fichier `.env.example` documente la liste attendue sans contenir les valeurs réelles. Un fichier `.env` est requis localement et sur le VPS, placé hors arborescence Git.

Persistance des données. La base PostgreSQL applique les bonnes pratiques standard : un utilisateur applicatif aux droits limités, des sauvegardes quotidiennes planifiées sur le VPS, des migrations versionnées via Prisma qui interdisent les modifications de schéma manuelles en production.

Authentification utilisateur. Côté application web, Better-Auth gère les sessions DB-backed avec un identifiant opaque rotated régulièrement, et le mot de passe est hashé avec Argon2id. Le détail de cette implémentation est documenté dans le chapitre 5.

Ces précautions forment un socle défensif raisonnable pour un MVP. Le chapitre 7 détaille ce qu'il faudra renforcer en V2 : signature HMAC sur les routes internes, politique de rétention RGPD écrite noir sur blanc, et audit automatisé des dépendances.

Chapitre 4

Implémentation du moteur agent

Ce chapitre détaille le moteur agent : la partie qui orchestre la collecte, l'évaluation, l'analyse et la rédaction des rapports. Tout son code tient dans le répertoire `apps/agent` du monorepo, et il tourne comme un conteneur Docker autonome, accessible depuis l'application web sur un réseau privé.

4.1 Infrastructure Docker

Le moteur agent est livré sous la forme d'un service Docker dans le fichier `infra/docker/stack-complet`. Trois services coexistent dans la stack : le service `postgres` (PostgreSQL 17), le service `openclaw` (image officielle `ghcr.io/openclaw/openclaw:latest`) et le service `web` qui porte l'application Next.js. Les trois services partagent un réseau Docker interne nommé `radar-net`, ce qui leur permet de communiquer par leur nom de service sans passer par l'extérieur.

Le service OpenClaw expose le port 18789, mais seulement vers le réseau interne. Les variables d'environnement `DEEPSEEK_API_KEY`, `TAVILY_API_KEY` et `OPENCLAW_INTERNAL_SECRET` sont passées au conteneur depuis le fichier `.env` placé à la racine du projet. Le volume `apps/agent/openclaw-data` est monté dans le conteneur à l'emplacement `/data`, ce qui permet à OpenClaw d'y trouver sa configuration (`openclaw.json`), ses profils d'authentification (`auth-profiles.json`) et les fichiers `SKILL.md` des huit compétences. Le volume `apps/agent/workspace` est monté de la même manière pour rendre l'arborescence des skills modifiable sans rebuild du conteneur.

La commande de démarrage typique depuis le dossier de la stack est la suivante :

```
1 cd infra/docker/stack-complet
2 docker compose --env-file "../../.env" up -d
```

Listing 4.1. Commande de démarrage de la stack de développement

4.2 Configuration OpenClaw

Toute la configuration d'OpenClaw tient dans deux fichiers situés dans `apps/agent/openclaw-data`. Le premier, `openclaw.json`, décrit la structure du moteur : agents déclarés, modèles

disponibles, providers de recherche web, paramètres de scheduling et timeouts par défaut. Le second, `auth-profiles.json`, déclare les profils d'authentification auprès des fournisseurs externes en mappant des clés symboliques sur les variables d'environnement.

Le fichier `openclaw.json` déclare un seul agent nommé `main`. Cet agent charge tous les skills du dossier `workspace/skills`. Les paramètres globaux retiennent une attention particulière : `thinkingDefault` est positionné à `medium`, ce qui active le raisonnement intermédiaire de DeepSeek V4 Pro sans le pousser à son maximum (le mode `xhigh` multipliait par trois la durée des cycles pour un gain de qualité marginal sur nos tests). Le paramètre `timeoutSeconds` est fixé à 900 (quinze minutes), ce qui couvre largement la durée typique d'une compétence du pipeline. Le scheduling est configuré pour déclencher l'agent `main` chaque lundi à six heures du matin, fuseau `Africa/Casablanca`, mais en pratique nous laissons ce déclencheur côté application web pour faciliter la gestion multi-utilisateur.

L'extrait suivant illustre les blocs essentiels de `openclaw.json` :

```
1 {
2   "agents": {
3     "defaults": {
4       "model": "deepseek/deepseek-v4-pro",
5       "thinkingDefault": "medium",
6       "timeoutSeconds": 900,
7       "fastDefault": "off"
8     },
9     "list": [
10      { "id": "main", "skillsDir": "workspace/skills" }
11    ]
12  },
13  "plugins": {
14    "tavily": { "apiKeyEnv": "TAVILY_API_KEY" },
15    "duckduckgo": { "enabled": true }
16  },
17  "scheduler": {
18    "crons": [
19      { "agent": "main", "expr": "0 6 * * 1", "tz": "Africa/Casablanca"
20        }
21    ]
22  }
```

Listing 4.2. Extrait significatif de `openclaw.json`

4.3 Architecture skills versus agents

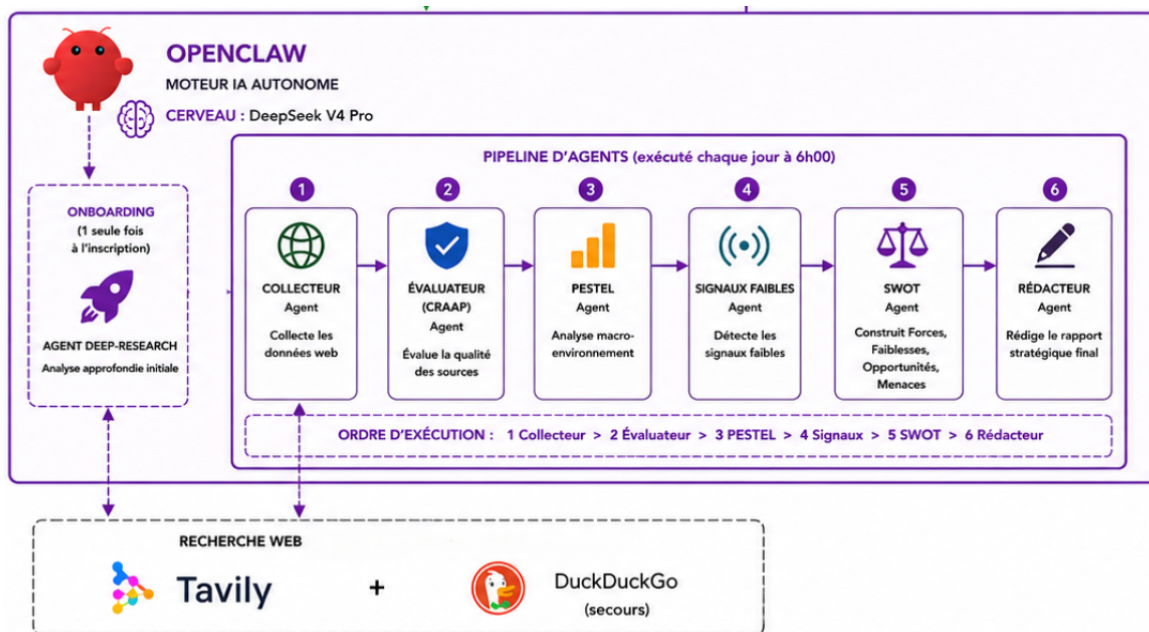


Figure 4.1. Configuration retenue : un agent chargeant huit compétences

Cette section explique la décision la plus structurante du projet : un seul agent qui charge huit compétences, plutôt que huit agents distincts.

Dans OpenClaw, un agent est une entité complète avec son modèle LLM, ses outils et sa mémoire. Une compétence (skill) est plus légère : un script de comportement décrit dans un fichier `SKILL.md` composé d'un en-tête YAML et d'un corps en Markdown libre. Le mode multi-agent d'OpenClaw permet de faire cohabiter plusieurs agents dans un même processus, mais il est conçu pour router des messages externes vers le bon agent, pas pour orchestrer un pipeline interne : la communication inter-agents passerait par des allers-retours HTTP coûteux qui dilueraient le contexte.

La fonction `sessions_spawn` répond exactement à notre besoin. Elle crée une sous-session du même agent avec un contexte vierge, ce qui correspond à la logique d'un pipeline séquentiel où chaque étape doit démarrer proprement avant d'alimenter la suivante.

Une dernière contrainte a tranché le débat : le YAML de `SKILL.md` ne permet pas de surclasser le modèle LLM ni le niveau de `thinking` par compétence. Après tests, un `thinking=medium` global s'est révélé satisfaisant pour toutes les étapes, ce qui retire l'unique raison qui nous aurait poussés vers le multi-agent.

4.4 Les huit compétences SKILL.md

Le pipeline de RADAR repose sur huit compétences spécialisées. Cette section présente chacune d'elles, son rôle dans le cycle, ses entrées et ses sorties.

4.4.1 Deep Research

Deep Research est la seule compétence du moteur qui s'exécute en dehors du pipeline orchestré. Elle est déclenchée une seule fois par utilisateur, au moment de l'inscription, et ne participe à aucun cycle de veille hebdomadaire. Son exécution est indépendante : elle n'est pas appelée par l'orchestrateur et n'alimente pas directement les sept autres compétences dans la même session ; son résultat est stocké en base pour que les cycles ultérieurs puissent le réutiliser.

Son rôle est de transformer un couple basique « nom d'entreprise + site web » en un profil enrichi exploitable par le reste du système : secteur d'activité, taille (**sizeBand**), ville du siège, produits principaux, clientèle cible (**ICP**), positionnement, mots-clés métier et concurrents potentiellement non listés par l'utilisateur.

Deep Research utilise les outils natifs d'OpenClaw **web_search** et **web_fetch**. Sur DuckDuckGo plutôt que sur Tavily, parce que ses requêtes sont espacées dans le temps (typiquement trois ou quatre requêtes sur cinq minutes) et qu'elle ne déclenche donc pas la détection de bot. Le SKILL.md décrit une procédure en quatre temps : recherche du site officiel, recherche d'informations annexes (presse, LinkedIn, registres d'entreprise), recoupement, et synthèse au format JSON attendu.

La sortie de Deep Research part directement sur **/api/internal/profil**, avec la structure **ProfilUtilisateur**. Le profil reste *permanent* en base : il est relu à chaque cycle et passé à l'orchestrateur dans le message initial. La raison est économique : Deep Research consomme environ trente mille tokens DeepSeek, et le profil d'une entreprise bouge peu d'une semaine à l'autre. Le relancer à chaque cycle n'aurait aucun sens.

4.4.2 Orchestrateur

L'orchestrateur est le chef d'orchestre du pipeline hebdomadaire. Il n'exécute aucune tâche propre : il enchaîne les sous-sessions via **sessions_spawn** et propage les résultats d'une étape à l'autre. Il est déclenché soit par le cron OpenClaw le lundi à six heures, soit par un appel HTTP venant de l'application web (bouton « Lancer un cycle maintenant » dans le dashboard).

L'entrée de l'orchestrateur est un message qui contient l'identifiant du rapport (**rapportId**), l'identifiant utilisateur (**userId**), le profil complet de l'entreprise utilisatrice (déjà produit par Deep Research et stocké en base) et la liste des concurrents

à surveiller. Cette charge utile est suffisamment volumineuse pour que nous ayons documenté sa structure dans le `SKILL.md` de l'orchestrateur, afin que les autres compétences sachent exactement ce qu'elles peuvent attendre lorsqu'elles sont invoquées.

La sortie principale de l'orchestrateur est un POST vers `/api/internal/rapport/termine` contenant le rapport final. En parallèle, après chaque étape, l'orchestrateur POST vers `/api/internal/rapport/progressse` pour notifier l'application web de l'avancement. En cas d'erreur (timeout, exception, retour incohérent d'une compétence), il POST vers `/api/internal/rapport/echec` avec le motif. Cette triple route permet à l'utilisateur de suivre la progression en temps réel dans son dashboard.

Une particularité importante découverte lors des tests : la valeur de `runTimeoutSeconds` passée à chaque appel de `sessions_spawn` écrase le `timeoutSeconds` global de la configuration OpenClaw. Le `SKILL.md` de l'orchestrateur fixe explicitement ce paramètre à 900 secondes pour chaque étape, sinon la valeur par défaut de 300 secondes s'appliquait et faisait échouer l'évaluateur ou les analystes sur les rapports denses.

4.4.3 Collecteur

Le collecteur est la première étape exécutive du pipeline hebdomadaire. Il reçoit en entrée le profil utilisateur et la liste des concurrents, et il doit produire un ensemble de sources fraîches couvrant les mouvements stratégiques des concurrents sur les sept derniers jours.

La logique du collecteur est décrite en quatre phases dans son `SKILL.md`. La phase 1 établit la liste des cibles à interroger : pour chaque concurrent, le site officiel cité explicitement dans le profil constitue une source primaire directe. La phase 2 invoque `web_search` avec Tavily, en formant pour chaque concurrent des requêtes ciblées sur les axes activées par l'utilisateur (recrutements, stratégie, technologie, présence digitale, conformité). La phase 3 collecte les URL retournées et leur contenu (Tavily renvoie le contenu complet des pages dans la réponse de l'API, pas seulement les liens). La phase 4 applique un filtrage temporel strict : toute source dont la date connue est antérieure à la fenêtre des sept derniers jours est écartée, sans exception de pertinence.

Le `SKILL.md` inclut une logique conditionnelle pour gérer le fallback. Si la clé `TAVILY_API_KEY` est présente dans l'environnement, le collecteur passe par Tavily. Si elle est absente, il bascule sur DuckDuckGo en limitant la cadence à trois requêtes maximum par batch et en effectuant `web_fetch` pour récupérer le contenu des pages (DuckDuckGo ne retourne que des URL). Cette logique conditionnelle a évité plusieurs heures de bot blocking lors des premières semaines de développement, avant que nous ne prenions une clé Tavily.

La sortie du collecteur est une liste JSON de sources, chaque source contenant URL, titre, date détectée, extrait textuel et concurrent associé. Cette liste est POSTée

vers `/api/internal/sources` pour persistance, et retournée à l'orchestrateur dans un bloc `json` explicite, ce qui permet à l'orchestrateur de la transmettre directement à l'évaluateur sans la reconstruire.

4.4.4 Évaluateur

L'évaluateur reçoit la liste des sources collectées et calcule pour chacune un score CRAAP détaillé. Les cinq critères (Currency, Relevance, Authority, Accuracy, Purpose) sont notés sur quatre points chacun, ce qui donne un score global sur vingt. Une source dont le score global descend en dessous de cinq sur vingt est marquée `rejetee=true` et n'alimente pas les analyses en aval.

Le traitement des sources se fait par lots de cinq pour éviter les timeouts. Cette décision est venue après un premier test où l'évaluateur tentait de scorer dix-neuf sources en un seul appel, ce qui dépendait massivement de la latence DeepSeek et finissait par excéder le timeout. Le batch de cinq combiné à un timeout de 900 secondes a stabilisé le composant : les cycles de tests ultérieurs ont tous abouti avec un score CRAAP moyen entre 14 et 16 sur 20 sur des cas réels.

L'évaluateur renvoie ensuite un POST vers `/api/internal/sources` : les sources enrichies du score CRAAP détaillé (chaque critère et le total), de la décision `rejetee` et d'une justification courte. C'est ce qui rend l'audit possible : n'importe qui peut refaire le calcul à la main sur une source et vérifier qu'il tombe juste.

4.4.5 Analyste PESTEL

L'analyste PESTEL analyse l'environnement sectoriel global plutôt qu'une organisation spécifique. Il segmente cet environnement en six dimensions (politique, économique, sociologique, technologique, environnementale, légale) et produit pour chacune un ensemble de facteurs influents.

Les entrées sont la liste des sources évaluées, filtrées par un seuil de score CRAAP supérieur ou égal à six sur vingt et un score Currency supérieur ou égal à deux sur quatre. Ce double filtre évite que des sources marginalement acceptables (CRAAP=6) mais très anciennes (Currency=1) ne polluent l'analyse. Le SKILL.md demande explicitement de mentionner pour chaque facteur PESTEL la dimension concernée, l'intensité de l'impact (faible, moyenne, forte), le sens (positif, négatif, neutre) et un ou plusieurs identifiants de sources qui le justifient.

La sortie part vers `/api/internal/pestel`. Chaque entrée rappelle sa dimension, si bien qu'une analyse PESTEL reste lisible même sortie de son contexte. Elle alimente ensuite l'étape suivante, l'analyste SWOT.

4.4.6 Analyste SWOT

L'analyste SWOT produit une grille classique en quatre quadrants : forces internes, faiblesses internes, opportunités externes, menaces externes. La grille porte sur l'entreprise utilisatrice et non sur les concurrents pris individuellement, car le M244 enseigne la SWOT comme un outil d'analyse *de l'organisation dans son environnement*, pas comme un benchmark concurrent par concurrent.

L'analyste reçoit trois choses : le profil utilisateur enrichi (issu de Deep Research), les sources évaluées (filtrées par $\text{CRAAP} \geq 6$ et $\text{Currency} \geq 2$) et, surtout, la grille PESTEL de l'étape précédente. La PESTEL nourrit les deux quadrants externes de la SWOT : chaque facteur sectoriel est repris du point de vue de l'entreprise utilisatrice, ce qui donne les opportunités et les menaces. Les forces et les faiblesses internes, elles, viennent du profil croisé avec les sources collectées sur l'entreprise. On garde ainsi le fil entre la lecture macro de l'environnement et l'analyse propre à l'entreprise.

La sortie de l'analyste est un POST vers `/api/internal/swot` avec un objet JSON structuré : une liste de forces, une liste de faiblesses, une liste d'opportunités, une liste de menaces, et un score de confiance global compris entre zéro et un. Chaque élément est accompagné d'une justification courte qui pointe vers les sources mobilisées.

4.4.7 Détecteur de signaux faibles

Cette compétence est la plus exigeante méthodologiquement. Elle ne cherche pas dans les sources les informations explicites, mais les indices précoces qui annoncent un mouvement de fond. Elle reçoit en entrée non seulement les sources du cycle courant, mais aussi les sources des trente derniers jours conservées en base.

Le SKILL.md demande au détecteur de raisonner sur des accumulations : un sujet qui revient sur plusieurs sources mineures (forums, blogs, offres d'emploi), une thématique qui prend de l'ampleur sans avoir fait l'objet d'annonces officielles, un faisceau d'indices convergents qui pointe vers une décision stratégique à venir. Pour chaque signal détecté, il produit une intensité (faible, moyenne, forte), un horizon temporel d'occurrence (court, moyen, long terme), un libellé, une description et la liste des sources qui le supportent.

La sortie est postée vers `/api/internal/signaux`. Contrairement aux analyses SWOT et PESTEL qui filtrent strictement par score CRAAP, le détecteur conserve les sources de score modérément bas (3 à 5) parce que la veille des signaux faibles passe nécessairement par des sources de qualité mixte. Le SKILL.md le mentionne explicitement pour éviter de reproduire le filtre des analystes.

4.4.8 Rédacteur

Le rédacteur reçoit en entrée la totalité des sorties précédentes : profil utilisateur, sources évaluées, grille PESTEL, grille SWOT, liste de signaux faibles. Il produit un rapport narratif structuré qui suit le plan académique enseigné dans le module M244 : synthèse exécutive, méthodologie et collecte, analyse, résultats, recommandations stratégiques.

Le SKILL.md du rédacteur impose une structure en six sections avec des contraintes de longueur indicatives : entre cent et cent cinquante mots pour la synthèse exécutive, et entre mille et mille cinq cents mots pour le rapport complet. Le ton est neutre, factuel, pas commercial. La sortie inclut également cinq recommandations stratégiques chiffrées (priorité, effort, impact attendu) et un score de confiance global sur le cycle.

Le rédacteur ne POST pas son rapport vers une route spécifique. Il retourne le rapport complet à l'orchestrateur qui se charge du POST vers `/api/internal/rapport/termine` pour clôturer le cycle. Cette délégation évite que le rédacteur n'ait à connaître les conventions de l'API interne et le maintient dans son rôle de « générateur de texte ».

4.5 Intégration de DeepSeek V4 Pro

Le choix du modèle s'est fait après une série de tests. GPT-4o nous servait de référence : tout modèle aussi bon ou meilleur que lui était acceptable. Le tableau 4.1 résume les critères qui nous ont conduits à retenir DeepSeek V4 Pro.

Table 4.1. Comparatif des modèles LLM évalués pour RADAR

Critère	GPT-4o	Claude Opus 4.7	DeepSeek V4 Pro
Contexte maximal	128k tokens	200k tokens	1M tokens
Coût entrée (par M tokens)	5 \$	15 \$	0,30 \$
Coût sortie (par M tokens)	15 \$	75 \$	1,20 \$
Limite de débit	30k TPM (Tier 1)	modérée	élastique
Qualité analyses SWOT/PESTEL	Bonne	Excellente	Très bonne
Compatibilité API OpenAI	Native	Non	Native
Adapté au budget académique	Non	Non	Oui

GPT-4o a été écarté sur le couple débit/coût : sa limite de 30k tokens par minute au Tier 1 saturait dès le premier appel de Deep Research (environ 28k tokens), et un cycle complet aurait coûté près d'un dollar, soit plusieurs centaines de dollars par mois pour quelques comptes de test seulement. Claude Opus 4.7 produit de meilleures

analyses, mais à quinze dollars le million de tokens d'entrée, il reste hors de portée d'une plateforme livrée gratuitement. DeepSeek V4 Pro réunit ce qu'il nous fallait : un contexte d'un million de tokens, un coût dix-sept fois inférieur à GPT-4o, un débit élastique et une compatibilité native avec l'API OpenAI d'OpenClaw. L'intégrer n'a demandé qu'un changement d'URL et de clé dans `auth-profiles.json`.

4.6 Intégration de Tavily et fallback DuckDuckGo

Le moteur de recherche web est essentiel au collecteur : sans lui, RADAR n'a rien à évaluer. Nous avons d'abord testé DuckDuckGo, gratuit et sans clé API. Problème : il bloque l'adresse IP dès qu'on émet trop de requêtes rapprochées, et le collecteur en envoie quinze à vingt en moins d'une minute, ce qui déclenchait systématiquement le blocage.

Nous sommes donc passés à Tavily, un moteur de recherche conçu pour les agents IA. Il offre mille requêtes mensuelles gratuites, retourne directement le contenu complet des pages dans sa réponse JSON, et n'applique pas de limite de cadence. La consommation typique de RADAR (environ quarante requêtes par cycle) reste largement dans cette enveloppe.

Le `SKILL.md` du collecteur gère le basculement automatiquement : si la clé `TAVILY_API_KEY` est présente, il utilise Tavily ; sinon il revient sur DuckDuckGo avec des requêtes ralenties (batchs de trois) et un `web_fetch` complémentaire pour récupérer le contenu des pages. Deep Research, dont les requêtes sont naturellement espacées, reste sur DuckDuckGo pour économiser le quota Tavily.

4.7 Communication entre agent et application web

La communication entre le moteur agent et l'application web se fait intégralement en HTTP, sur le réseau Docker interne. Chaque compétence et l'orchestrateur lui-même effectuent des POST vers des routes nommées `/api/internal/*` portées par l'application Next.js. Ces routes sont décrites en détail dans le chapitre 5.

Le tableau 4.2 synthétise les routes attendues et leur utilisation côté agent.

Table 4.2. Routes internes consommées par le moteur agent

Route	Appelant	Charge utile principale
POST /api/internal/profil	Deep Research	Profil enrichi de l'entreprise utilisatrice
POST /api/internal/sources	Collecteur (création) puis Évaluateur (mise à jour CRAAP)	Liste des sources avec métadonnées et scores
POST /api/internal/pestel	Analyste PESTEL	Grille PESTEL six dimensions avec intensité et sens
POST /api/internal/swot	Analyste SWOT	Grille SWOT structurée avec score de confiance
POST /api/internal/signaux	Détecteur	Liste des signaux faibles avec intensité et horizon
POST /api/internal/rapport/progresses	Orchestrateur (après chaque étape)	Avancement du cycle pour notification dashboard
POST /api/internal/rapport/termine	Orchestrateur (fin de cycle)	Rapport final, recommandations, score global
POST /api/internal/rapport/echec	Orchestrateur (en cas d'erreur)	Motif d'échec, étape concernée, contexte

Toutes les requêtes sont en JSON. L'identifiant du rapport (**rapportId**) accompagne chaque appel pour assurer la cohérence : c'est cet identifiant qui sert de clé de regroupement en base de données. La réponse attendue est un objet `{ success: true }` si tout s'est bien passé, ou un objet `{ success: false, error: <message> }` dans le cas contraire. Le SKILL.md de chaque compétence interprète la réponse et lève une exception si **success** vaut **false**, ce qui propage l'erreur à l'orchestrateur.

Chaque compétence fait deux choses à la fois : un POST de persistance, et un retour direct du résultat à l'orchestrateur pour alimenter l'étape suivante. C'est ce que nous appelons le « principe hybride » dans notre documentation interne. On garde ainsi la visibilité temps réel dans le dashboard et la fluidité du pipeline, sans attendre une relecture en base.

4.8 Gestion des timeouts et fiabilité du pipeline

Un pipeline d'agents qui appelle des services externes pendant près d'une heure ne tient pas sans stratégie explicite de gestion des erreurs et des timeouts. Nous avons travaillé sur plusieurs fronts.

Le batching côté évaluateur est la première précaution prise. Comme décrit en section 4.4.4, le scoring CRAAP est effectué par lots de cinq sources. Cette taille de lot résulte d'un compromis : trop petit, le nombre d'appels DeepSeek explose et la latence cumulée allonge le cycle ; trop grand, chaque appel risque le timeout. Cinq sources par lot s'est révélé être le point d'équilibre après itérations.

La gestion fine des timeouts est venue plus tard, après un debug coûteux. Comme mentionné en section 4.4.2, le paramètre `runTimeoutSeconds` passé à chaque `sessions_spawn` écrase le `timeoutSeconds` global. Nous avons fixé ce paramètre à 900 secondes (quinze minutes) pour chaque étape, ce qui couvre largement le pire cas observé (l'évaluateur sur dix-neuf sources, environ douze minutes). Le diagnostic de cette subtilité a coûté deux cycles de test échoués avant d'être identifié : sans cette consigne explicite, le timeout par défaut de 300 secondes faisait échouer systématiquement les étapes d'analyse les plus denses.

Reste la traçabilité des erreurs. Chaque exception levée dans une compétence ou dans l'orchestrateur produit un POST vers `/api/internal/rapport/echec` avec un motif lisible, l'identifiant du rapport et l'étape concernée. Côté serveur, ce POST crée un enregistrement `EvenementRapport` de type `ECHEC` dans la table dédiée. L'utilisateur voit alors dans son dashboard l'état « échoué » avec le motif, plutôt qu'un cycle qui reste bloqué en « en cours » indéfiniment.

Ces précautions cumulées ont stabilisé le pipeline au point que les trois derniers cycles de test (rapport-test-005 inclus) se sont tous terminés avec succès en moins d'une heure, sur des cas réels et avec un score CRAAP moyen supérieur à 16 sur 20.

Chapitre 5

Implémentation de l'application web

Ce chapitre documente l'application web, qui est le point d'entrée unique de RADAR : elle gère l'authentification, guide l'onboarding, conditionne l'accès au tableau de bord et stocke l'ensemble des données. La stack (front Next.js 16, React 19 et Tailwind CSS, adossés à PostgreSQL via Prisma) a déjà été justifiée au chapitre 3 ; ici, nous nous concentrons sur les parcours fonctionnels et leur implémentation.

5.1 Authentification et gestion des utilisateurs

La gestion de l'identité repose sur **Better Auth**, une bibliothèque d'authentification intégrée nativement au App Router de Next.js, adossée à la base PostgreSQL via Prisma.

Lorsqu'un visiteur arrive pour la première fois sur la plateforme, il dispose de deux manières de se créer un compte :

- **Inscription classique** par adresse électronique et mot de passe : l'utilisateur renseigne son nom, son adresse et un mot de passe (entre 8 et 128 caractères). Le mot de passe n'est jamais stocké en clair, il est haché avec l'algorithme **Argon2id**.
- **Connexion par Google** (OAuth) : en un clic, l'utilisateur s'authentifie avec son compte Google, sans créer de mot de passe dédié.

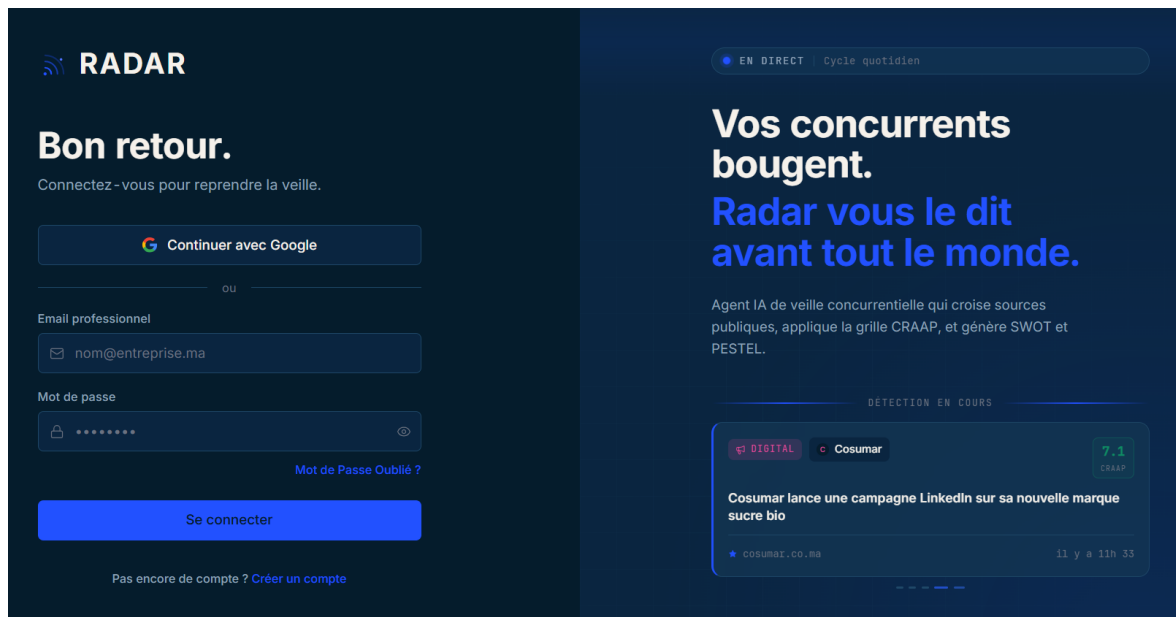


Figure 5.1. Page de connexion

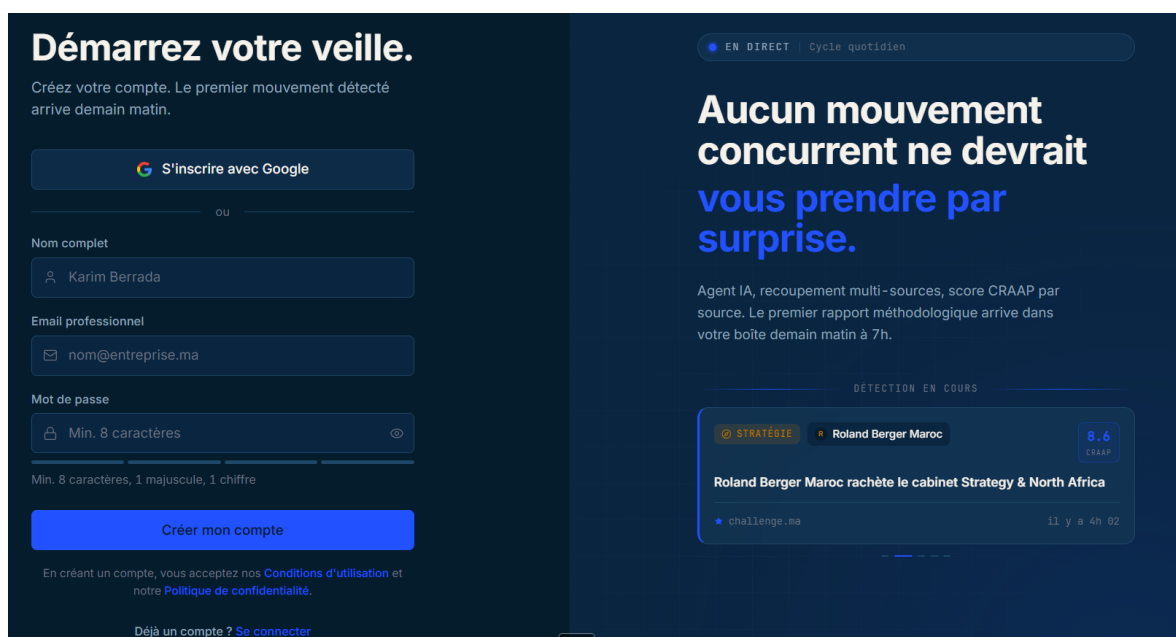


Figure 5.2. Création de compte

5.2 Onboarding guidé en trois étapes

Une fois le compte créé, comme c'est sa toute première venue, l'utilisateur est dirigé vers un **parcours d'onboarding guidé en trois étapes**. Ce parcours est indispensable : on ne peut pas conduire une veille pertinente sur une entreprise sans connaître son contexte. Le système suit une logique de machine à états qui aiguille automatiquement l'utilisateur vers la prochaine étape non remplie (profil, puis concurrents, puis axes), et le mène au tableau de bord une fois tout complété.

Étape 1 : profil de l'entreprise. L'utilisateur renseigne deux informations : le **nom de son entreprise** et son **site Internet**. C'est le socle minimal à partir duquel tout le reste est déduit.

RADAR

1 — 2 — 3

Votre entreprise Vos concurrents Vos axes

Parlons de votre entreprise.

Nom de votre entreprise

Marjane

Site internet

https://www.marjane.ma/

AI Deep Research

Dès que vous passez à l'étape suivante, notre agent autonome explore le web pour reconstituer votre profil business — secteur, positionnement, concurrents implicites et signaux stratégiques récents.

SECTEUR CONCURRENTS POSITIONNEMENT SIGNALS RÉCENTS

Suivant →

Figure 5.3. Onboarding, étape 1 : profil de l'entreprise (nom et site web)

Dès la validation de l'étape 1, une **recherche automatique se déclenche en arrière-plan** pour caractériser le contexte de l'entreprise : son secteur d'activité, ses produits et services, ses marchés, son positionnement et sa clientèle cible. Cette analyse alimente le profil sans bloquer l'utilisateur, qui passe immédiatement à l'étape suivante pendant que le traitement se poursuit. C'est une étape déterminante : sans cette compréhension du contexte, aucune veille concurrentielle compréhensive n'est possible.

Étape 2 : concurrents à surveiller. L'utilisateur indique les concurrents qu'il souhaite suivre (nom et, si disponible, site web). Le point fort ici est que la recherche d'arrière-plan a déjà **identifié et suggéré des concurrents** : lorsqu'elle a abouti, ces suggestions apparaissent et peuvent être ajoutées en un clic. L'utilisateur conserve toutefois la main et peut ajouter librement ses propres concurrents. Au moins un concurrent est requis pour continuer.

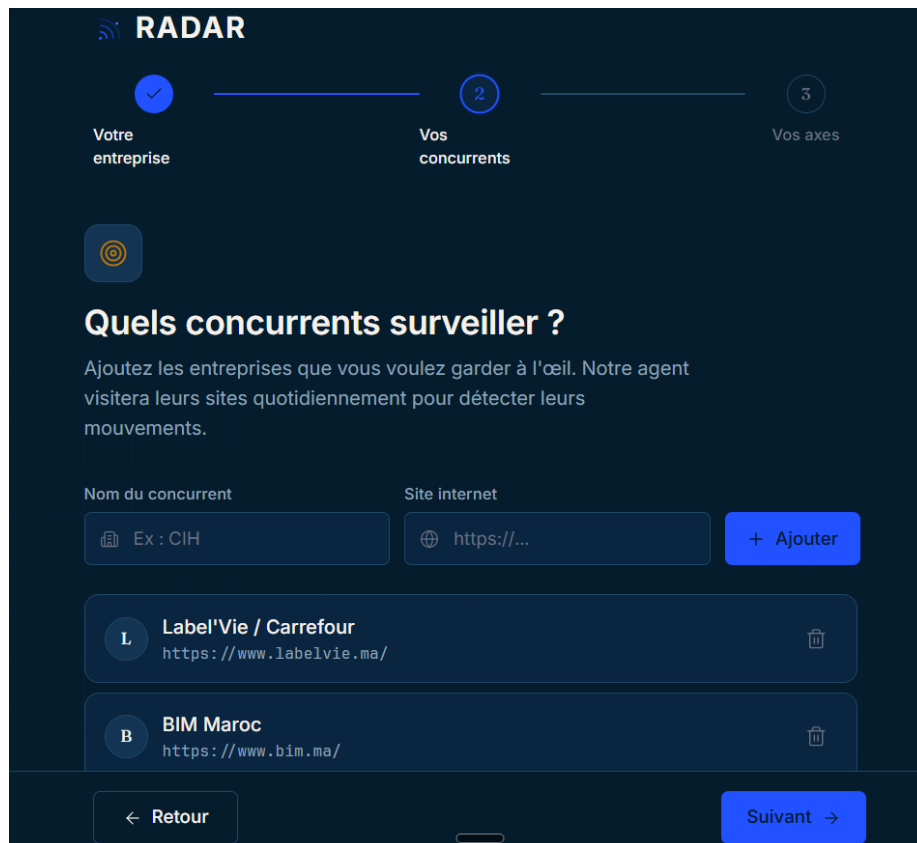


Figure 5.4. Onboarding, étape 2 : saisie des concurrents et suggestions automatiques

Étape 3 : axes de surveillance. L'utilisateur sélectionne les signaux qui l'intéressent parmi cinq axes : recrutement et RH, stratégie et direction, technologie et innovation, présence digitale, et réglementation et conformité. Cette sélection oriente ensuite la collecte et l'analyse du moteur agent. La validation de cette étape marque la fin de l'onboarding (horodatage d'achèvement en base) et ouvre une page de confirmation récapitulant le profil enrichi, les concurrents et les axes retenus.



Figure 5.5. Onboarding, étape 3 : sélection des axes de surveillance



Figure 5.6. Page de confirmation de fin d'onboarding

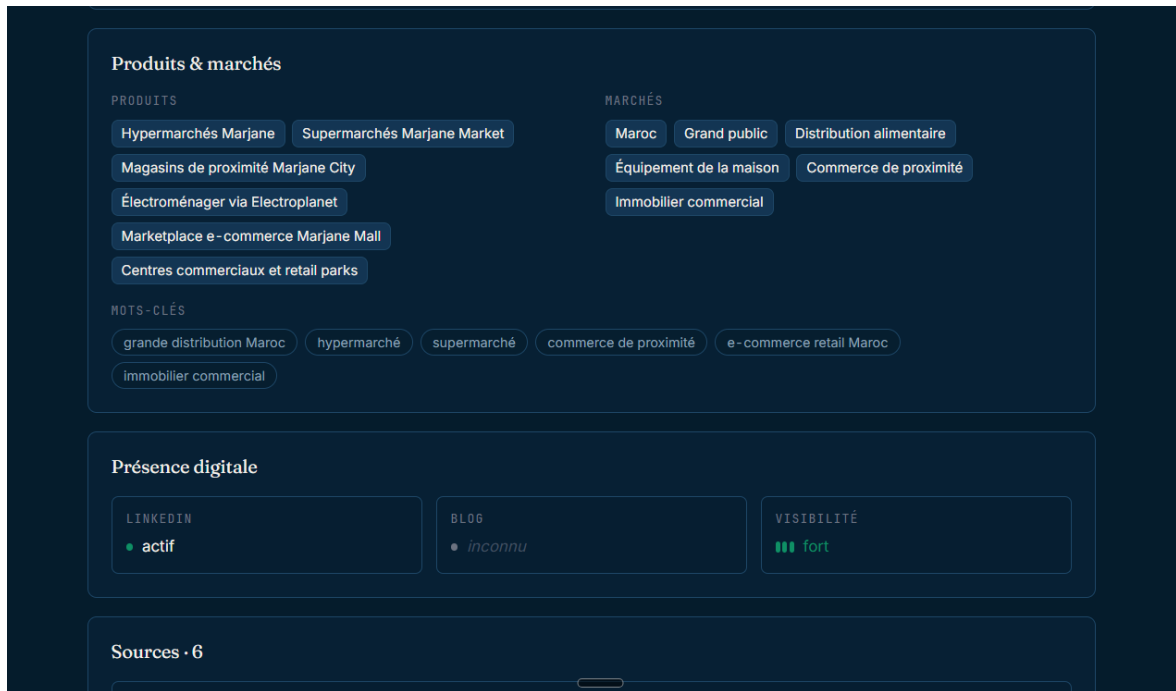


Figure 5.7. Résultat de la recherche d'arrière-plan (Deep Research) sur l'entreprise

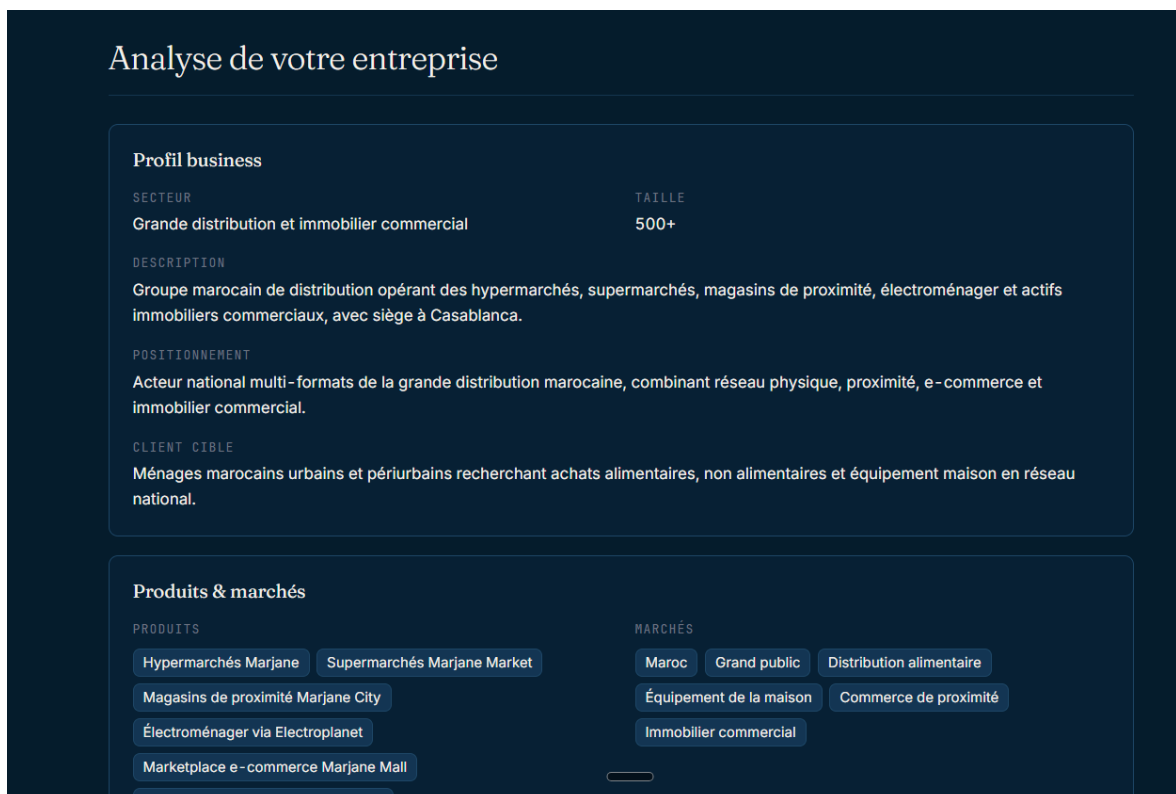


Figure 5.8. Résultats détaillés de la recherche sur l'entreprise présentés à la fin de l'onboarding

5.3 Garde d'accès et condition d'onboarding

L'accès au tableau de bord est **conditionnel**. À chaque connexion, l'application vérifie systématiquement si l'utilisateur a terminé son onboarding. Cette garde est portée par le layout du groupe de routes applicatives : si l'onboarding n'est pas complété, l'utilisateur est automatiquement redirigé vers le parcours d'onboarding, et il ne peut donc accéder au tableau de bord que lorsque les trois étapes sont validées. L'ensemble de ces données (profil, enrichissement, concurrents, axes) est stocké dans la base PostgreSQL gérée via Prisma.

5.4 Le tableau de bord

Une fois l'onboarding complété, l'utilisateur accède au tableau de bord, organisé autour d'une barre de navigation latérale en trois groupes : « Aujourd'hui » (Brief et Signaux faibles), « Analyse » (Concurrents, SWOT, PESTEL) et « Livrables » (Rapports, Cycles), plus les Paramètres. Les pages principales sont les suivantes :

- **Brief** : la vue d'entrée, sous forme de tableau Kanban présentant l'avancement hebdomadaire des rapports concurrent par concurrent.
- **Signaux faibles** : le flux des signaux faibles détectés par croisement sur une fenêtre glissante de trente jours.
- **Concurrents** : le hub de gestion des concurrents suivis (ajout, modification, suppression).
- **SWOT** : la matrice SWOT par concurrent, versionnée par semaine.
- **PESTEL** : l'analyse PESTEL en six dimensions, versionnée par semaine.
- **Rapports** : la bibliothèque des synthèses produites par l'agent, consultables dans l'application et exportables en PDF.
- **Cycles** : la traçabilité des cycles de veille exécutés par l'agent.
- **Paramètres** : la configuration du profil, des concurrents et des axes.

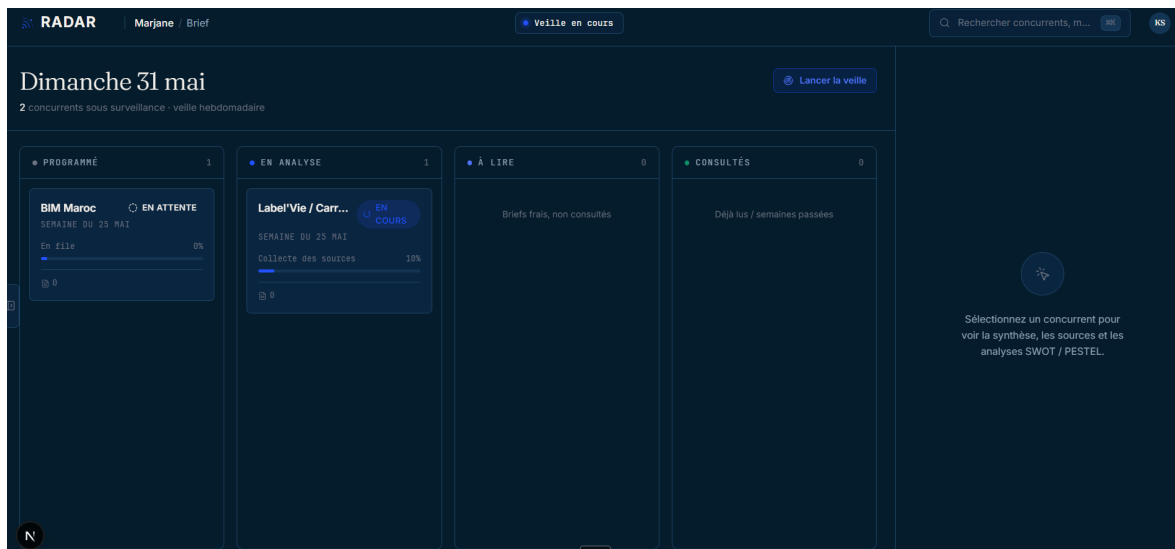


Figure 5.9. Page Brief : lancement de la veille depuis le tableau de bord

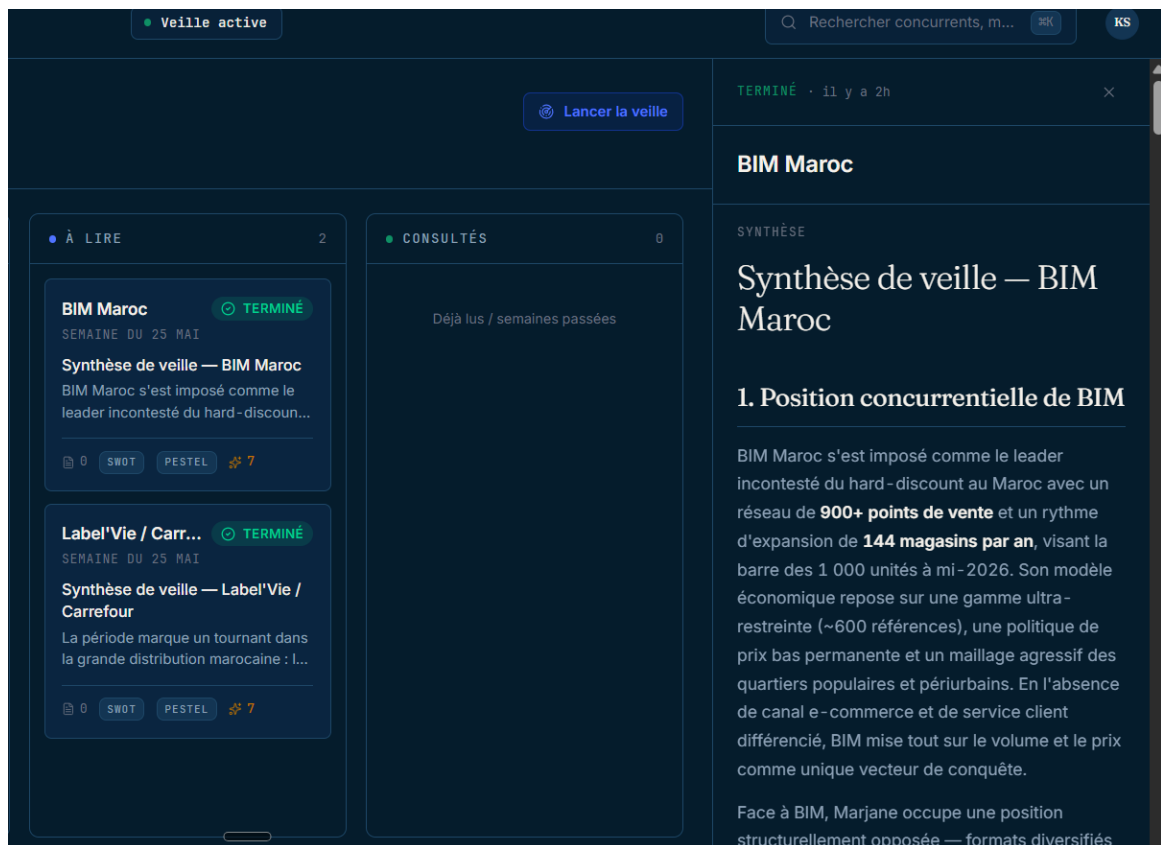


Figure 5.10. Page Brief : veille terminée



Figure 5.11. Page Signaux faibles

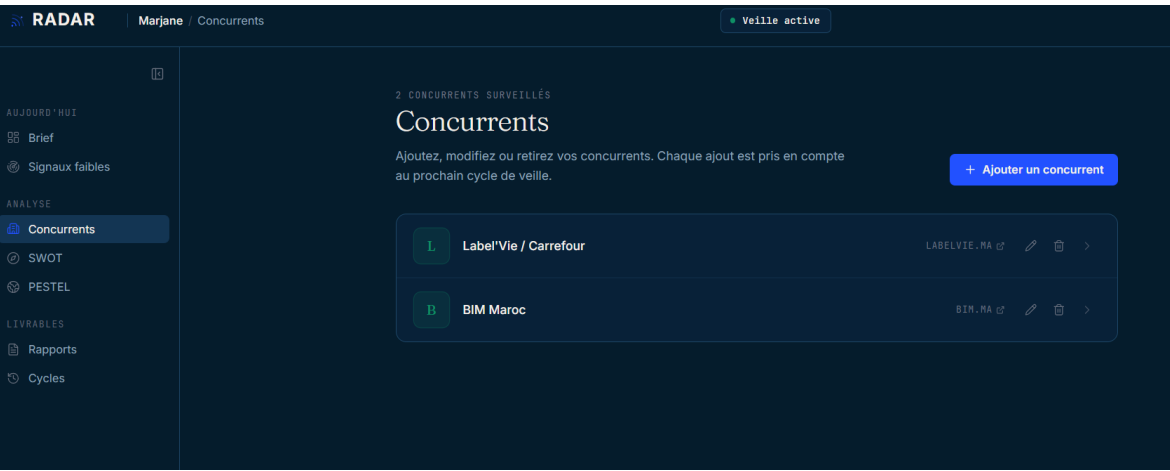


Figure 5.12. Page Concurrents



Figure 5.13. Page SWOT (dernière veille en date)



Figure 5.14. Page PESTEL



Figure 5.15. Page Rapports

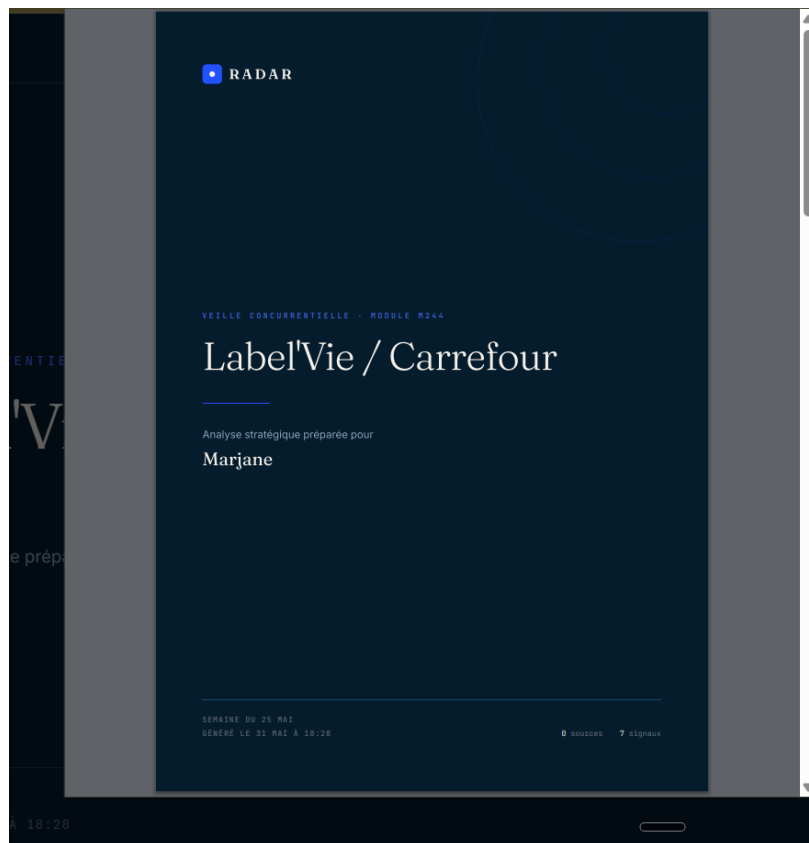


Figure 5.16. Export PDF d'un rapport de veille



Figure 5.17. Page Cycles de veille

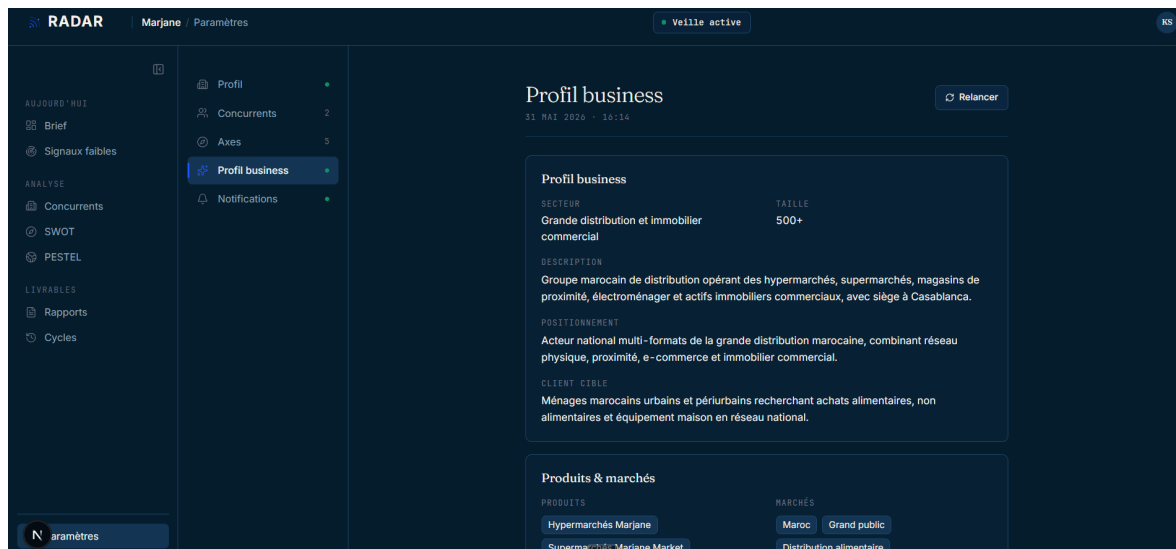


Figure 5.18. Page Paramètres



Figure 5.19. Paramètres : profil de l'entreprise connectée

5.5 Modèle de données et persistance

L'ensemble des données est persisté dans PostgreSQL via l'ORM Prisma. Les tables de l'identité (utilisateur, session, compte, vérification) sont gérées par Better Auth. Les tables métier portent le profil de l'entreprise et son enrichissement (**ProfilUtilisateur**), les concurrents suivis (**Concurrent**), ainsi que les rapports de veille et leurs composants : sources évaluées CRAAP, grilles SWOT et PESTEL, et signaux faibles. Cette base est la mémoire centrale partagée entre l'application web et le moteur agent, qui y écrit ses résultats via les routes internes décrites au chapitre 4.

Chapitre 6

Tests et validation du système

Ce chapitre rend compte de la démarche de tests menée pendant le projet.

6.1 Tests unitaires

Les tests unitaires valident le moteur agent en isolation, sans dépendre de l'application web. Ils s'appuient sur la stack Docker du moteur, où un mock-api Node.js remplace l'application web : il expose les huit routes `/api/internal/*` et enregistre chaque appel reçu dans `apps/agent/openclaw-data/tests/`, dans un sous-dossier par compétence appelante. Les fichiers sont horodatés au format ISO, de sorte qu'on peut rejouer la chronologie complète d'un cycle à partir des seules archives. Tout est reproductible et auditable : un évaluateur extérieur peut relancer un cycle et comparer les fichiers produits à ceux versionnés dans le dépôt.

6.1.1 Cas Marjane : test isolé de Deep Research

Le premier test a porté sur la compétence Deep Research, exécutée seule sur le cas Marjane, distributeur généraliste marocain choisi pour la richesse et l'hétérogénéité de sa présence en ligne. La compétence a produit en environ quatre minutes un profil de huit kilooctets de JSON contenant : le secteur (distribution généraliste), la taille (plus de 5000 employés), le siège (Casablanca), une liste de douze enseignes ou produits principaux, un descriptif de la clientèle cible, un positionnement stratégique, quinze mots-clés métier et cinq concurrents probables. L'exécution s'est faite via DuckDuckGo, sans déclencher de détection de bot.

6.1.2 Cycle complet : rapport-test-005

Le test du cycle complet, identifié `rapport-test-005`, a été exécuté le 7 mai 2026 entre 22h59 et 23h45. Il s'agit du premier cycle propre avec Tavily comme provider primaire. Les résultats sont résumés dans le tableau 6.1.

Table 6.1. Résultats du cycle de test `rapport-test-005` (mai 2026)

Métrique	Valeur observée
Provider de recherche utilisé	Tavily (clé dans <code>TAVILY_API_KEY</code>)
Nombre de sources collectées	20
Score CRAAP moyen après évaluation	16,15 / 20
Nombre de sources rejetées (<code>CRAAP < 5</code>)	0
Facteurs PESTEL identifiés	21 (répartis sur les 6 dimensions)
Forces SWOT identifiées	6
Faiblesses SWOT identifiées	5
Opportunités SWOT identifiées	6
Menaces SWOT identifiées	6
Signaux faibles détectés	8 (3 intensité forte, 4 moyenne, 1 faible)
Recommandations stratégiques produites	5
Score de confiance global	0,82
Durée totale du cycle	46 minutes
Taille du rapport narratif final	1400 mots structurés en 6 sections

Tous les POST attendus vers les routes internes ont retourné `{success: true}`. Le rapport final, archivé dans `tests/orchestracteur/termine-2026-05-07T23-45.json`, respecte la structure du cycle M244 et propose des recommandations chiffrées et priorisées.

6.1.3 Échecs rencontrés et corrections

Deux cycles ont échoué avant le succès de `rapport-test-005`, et leurs causes ont nourri directement les corrections du moteur.

rapport-test-003. L'évaluateur a dépassé le timeout de 300 secondes en tentant de scorer dix-neuf sources d'un coup, et le collecteur n'avait pas retourné sa liste dans un bloc JSON exploitable. Corrections appliquées : batching par lots de cinq sources côté évaluateur, et exigence explicite de « Réponse finale OBLIGATOIRE avec bloc "json" » dans le SKILL.md du collecteur.

rapport-test-004. L'évaluateur a fonctionné correctement, mais l'analyste PESTEL a dépassé le timeout. Le diagnostic a révélé que le `runTimeoutSeconds` par défaut (300 secondes) écrasait silencieusement le `timeoutSeconds=900` global lorsqu'il n'était pas déclaré explicitement dans chaque appel `sessions_spawn`. Correction : fixation à 900 secondes dans tous les appels `sessions_spawn` de l'orchestracteur.

Ces échecs ont également révélé que DeepSeek conserve le contexte d'une conversation précédente dans la même session OpenClaw, ce qui faisait remonter d'anciennes sources DuckDuckGo dans des tests Tavily. Le protocole de test a donc été ajusté :

nouvelle conversation à chaque lancement, et nettoyage de `openclaw-data` entre les cycles. La production de fichiers `echec-*.json` par ces deux cycles a aussi confirmé que la gestion d’erreur du pipeline fonctionne : l’utilisateur voit bien l’état d’échec dans son dashboard en cas de défaillance.

6.2 Tests d’intégration

Une fois le moteur agent connecté à l’application web, un premier test d’intégration de bout en bout a été conduit le **30 mai 2026** sur un compte réel. Contrairement aux tests unitaires du moteur, ce test valide la chaîne complète en conditions de production : déclenchement depuis le tableau de bord, exécution des compétences par OpenClaw, persistance via les routes `/api/internal/*`, stockage dans la base PostgreSQL hébergée sur Neon, puis restitution dans l’interface utilisateur.

Contexte du test. Le compte de test correspond à l’entreprise **inwi** (opérateur télécoms marocain, site `inwi.ma`), avec cinq axes de surveillance activés (stratégie et direction, technologie et innovation, présence digitale, réglementation et conformité, recrutement et RH) et deux concurrents suivis : **Orange Maroc** et **Maroc Telecom**. Le cycle hebdomadaire, déclenché manuellement depuis le dashboard, a traité les deux concurrents l’un après l’autre, conformément à la logique de file d’attente implémentée (un rapport par concurrent). La durée totale du cycle a été d’environ **70 minutes** pour les deux concurrents.

Résultats observés. Les deux rapports ont été produits, persistés et affichés dans le dashboard. Le tableau 6.2 en résume les métriques.

Table 6.2. Métriques du premier test d’intégration de bout en bout (compte inwi, 30 mai 2026)

Métrique	Rapport Orange Maroc	Rapport Maroc Telecom
Statut final	Terminé (100 %)	Terminé (100 %), avec incident partiel
Sources collectées	15	15
Domaines distincts	13 sur 15	12 sur 15
Score CRAAP moyen	16,40 / 20	16,40 / 20
Grille SWOT	Complète (5 forces, 5 faiblesses, 5 opportunités, 6 menaces)	Partielle (3 forces, 3 faiblesses, 4 opportunités, 5 menaces)
Grille PESTEL	Complète (19 facteurs sur les 6 dimensions)	Non produite (étape échouée)
Signaux faibles	7	6
Synthèse narrative	Produite	Produite

La diversité des sources confirme le respect du critère M244 de variété : presse économique (`lematin.ma`, `medias24.com`, `leseco.ma`, `telquel.ma`, `le360.ma`, `jeuneafrique.com`), communiqués officiels (`corporate.orange.ma`, `iam.ma`) et régulateur sectoriel (`anrt.ma`). La grille CRAAP est bien renseignée dimension par dimension : par exemple, la source « Orange Maroc signe une année record en 2025 » obtient Currency 2, Relevance 4, Authority 3, Accuracy 3, Purpose 4, soit un total de 16 sur 20.

Conformément à la méthodologie, les analyses portent sur l'entreprise utilisatrice (inwi) lue dans son environnement concurrentiel, et non sur chaque concurrent isolément. Le moteur a par exemple identifié comme force d'inwi son leadership silencieux du marché mobile (34,12 % de parts de marché) et le partenariat d'infrastructure FiberCo/TowerCo avec Maroc Telecom dont Orange est exclu. Parmi les signaux faibles les plus pertinents : « Isolement infrastructurel d'Orange » (pertinence 10 sur 10), « FiberCo/TowerCo : un duopole d'infrastructure qui marginalise Orange » (9 sur 10) et « Course aux Super Apps » (9 sur 10).

Incident rencontré et robustesse. Sur le second concurrent (Maroc Telecom), l'étape d'analyse PESTEL/SWOT a partiellement échoué : la grille PESTEL n'a pas été persistée et la SWOT est restée incomplète. Le pipeline a néanmoins poursuivi son exécution, consigné l'erreur (« Étape PESTEL/SWOT échouée pendant le cycle ») et livré malgré tout la synthèse narrative et les signaux faibles du concurrent. Ce comportement valide la stratégie de gestion d'erreur décrite au chapitre 4 : une défaillance d'une étape sur un concurrent ne bloque pas l'ensemble du cycle, et l'incident reste traçable dans le rapport plutôt que de laisser le cycle bloqué silencieusement.

Bilan. Ce premier test d'intégration valide la connexion de bout en bout des deux parties du système et la conformité des livrables au cycle M244 (sources évaluées CRAAP, SWOT, PESTEL, signaux faibles, synthèse). Il met aussi en évidence un point de fiabilité à renforcer sur l'étape d'analyse, qui fera l'objet d'un correctif (reprise automatique de l'étape PESTEL/SWOT en cas d'échec partiel). Les tests de charge et de performance (plusieurs utilisateurs actifs en parallèle sur la même instance OpenClaw) ainsi qu'un test utilisateur sur un panel restreint de trois à cinq testeurs internes restent planifiés pour consolider cette validation.

Chapitre 7

Bilan et perspectives

Ce chapitre prend le recul nécessaire pour analyser le projet : ce qui fonctionne, ce qui ne fonctionne pas encore, les obstacles qui nous ont fait perdre du temps, ce que le travail nous a apporté. La dernière section ouvre sur la suite, depuis les corrections immédiates jusqu'aux extensions plus ambitieuses.

7.1 Analyse critique

7.1.1 Forces du projet

Ce qui frappe d'abord, c'est la cohérence méthodologique. Chaque élément du moteur agent renvoie à un concept du module M244 et à un fichier identifiable. Le cycle de veille en cinq étapes correspond aux compétences du pipeline, la grille CRAAP au scoring de l'évaluateur, les analyses SWOT et PESTEL aux deux compétences dédiées, les signaux faibles à la compétence du même nom : il est possible de remonter d'une fonctionnalité jusqu'au cours, et inversement de descendre d'un concept du cours jusqu'à son implémentation.

La modularité du moteur joue dans le même sens. Le format SKILL.md isole chaque compétence dans un fichier lisible en moins de dix minutes, et chaque compétence peut être testée indépendamment du reste du pipeline via le mock-api. Pour ajouter une nouvelle compétence (un détecteur de fake news en complément de CRAAP par exemple), il suffit d'écrire un SKILL.md et de le brancher sur l'orchestrateur, sans toucher aux compétences existantes. La même modularité ouvre une porte économique : on peut activer ou désactiver des compétences selon le plan tarifaire si le produit était commercialisé.

Côté organisation, le découpage en deux pôles (un pôle moteur agent, un pôle application web) avec un contrat d'interface clair (les routes `/api/internal/*`) nous a permis de travailler en parallèle sans se gêner mutuellement. Les fichiers de documentation interne du dépôt ont servi de mémoire partagée, ce qui aide énormément quand on revient sur le projet après une semaine d'absence.

Un point très important est le coût d'exploitation. La stack au complet (VPS Hetzner CX21 à 4 EUR par mois, Tavily sur le free tier, DeepSeek autour de 0,05 dollar par cycle) permet d'exploiter RADAR pour quelques dizaines d'utilisateurs sans facture salée. C'est une réalité économique rare pour une plateforme qui repose sur des grands

modèles de langage.

7.1.2 Faiblesses et limites de la V1

RADAR V1 n'est pas une plateforme production-ready. Plusieurs limites doivent être nommées clairement.

Le déploiement reste mono-VPS. Une défaillance matérielle, un redémarrage du serveur ou une saturation d'un container interrompt le service pour tous les utilisateurs. Il n'y a pas de haute disponibilité, pas de basculement automatique, pas de plan de reprise d'activité. Cette limite est explicitement acceptée dans le périmètre V1, mais elle exclut un usage commercial réel.

La quota Tavily est plafonnée à mille requêtes par mois sur le free tier. Avec une consommation typique de quarante requêtes par cycle hebdomadaire, cela permet d'exploiter le service pour environ vingt-cinq utilisateurs actifs en parallèle. Au-delà, il faut passer au plan payant ou industrialiser le fallback DuckDuckGo.

Les routes internes ne sont pas signées par HMAC. Le réseau Docker privé constitue une isolation suffisante en monoVPS, mais dès qu'un déploiement multi-nœud ou cloud managé interviendra, la signature deviendra indispensable. L'ajout n'est pas compliqué à faire, mais il n'est pas fait.

7.1.3 Difficultés rencontrées et solutions trouvées

Le tableau 7.1 synthétise les principales difficultés techniques que nous avons rencontrées et les corrections appliquées. Ce retour d'expérience constitue, à notre sens, une partie importante du livrable académique : il documente les pièges concrets à éviter pour quiconque voudrait reprendre la stack RADAR.

Table 7.1. Principales difficultés techniques et solutions apportées

Problème rencontré	Symptôme observé	Solution appliquée
Limite TPM GPT-4o Tier 1 (30k tokens/min)	Blocage dès le premier appel de Deep Research	Migration vers DeepSeek V4 Pro (contexte 1M, débit élastique)
Bot detection DuckDuckGo sur rafales	Cycles partiels, certains concurrents non interrogés	Migration vers Tavily comme primaire, DuckDuckGo en fallback
Confusion multi-agent vs sessions_spawn	Tentative initiale en multi-agent qui ne pouvait pas marcher pour un pipeline interne	Lecture approfondie de la doc OpenClaw, choix de 1 agent + 8 skills + sessions_spawn
runTimeoutSeconds local écrase le global	Évaluateur ou PESTEL en timeout silencieux à 300s alors que le global est 900s	Fixation explicite de runTimeoutSeconds=900 dans chaque appel sessions_spawn de l'orchestrateur
Phase 4 collecteur trop laxiste sur les dates	Sources hors fenêtre des 7 derniers jours dans les premiers rapports	Réécriture de la Phase 4 avec algorithme strict + filtre Currency >= 2 dans les analystes
auth-profiles.json refuse de démarrer si une clé env est absente	OpenClaw bloque au boot sans message clair	Ne déclarer dans auth-profiles.json que les profils dont la clé est effectivement présente
Mock-api ne distinguait pas collecteur vs évaluateur sur /api/internal/sources	Résultats mélangés dans le mauvais dossier de test	Détection sur la présence du champ scoreCRAAP dans le payload

Aucun de ces problèmes n'est insurmontable une fois identifié. Mais cumulés, ils représentent plusieurs jours de travail qui n'apparaissent ni dans le code final ni dans les schémas d'architecture. Le rapport tente de les rendre visibles parce qu'ils constituent une part substantielle de l'apprentissage réalisé pendant le semestre.

7.2 Apports du projet

Le projet nous a fait progresser sur une stack moderne, sur plusieurs plans : orchestrer des agents IA avec un runtime open source comme OpenClaw, déployer plusieurs services en Docker, brancher des API tierces (DeepSeek, Tavily, OpenAI) en composant

avec leurs quotas et leurs codes d'erreur, et monter un environnement de test reproductible. Le travail en binôme, autour d'un contrat d'interface explicite (les routes `/api/internal/*`), nous a imposé une discipline directement transférable en entreprise : tant que les payloads respectent le contrat, chaque pôle peut ignorer comment l'autre est fait à l'intérieur. Au-delà de la technique, le projet confronte une méthode théorique à une réalité qui force à trancher des questions que les cours laissent ouvertes : où placer le seuil CRAAP, quelle fenêtre retenir pour les signaux faibles, comment juger la fraîcheur d'une source. De quoi alimenter un dossier solide en entretien, en IA comme en data ou en stratégie.

7.3 Conclusion générale

RADAR répond à la question posée par le module M244 : comment transformer la méthodologie de veille stratégique enseignée en cours en un produit opérationnel qui automatise le cycle de bout en bout ? La réponse construite pendant le semestre n'est pas une démonstration académique, c'est un logiciel déployable qui collecte, évalue, analyse et synthétise chaque semaine des informations concurrentielles pour une entreprise donnée. Les tests d'intégration valident l'approche : sur le cas Marjane, le moteur a produit en moins d'une heure un rapport conforme au cycle M244, fondé sur vingt sources évaluées par CRAAP avec un score moyen supérieur à 16 sur 20.

7.4 Perspectives d'évolution

Plusieurs chantiers dépassent le périmètre de la V1 et constituent les pistes naturelles pour la suite.

Personnalisation des axes de surveillance. La V1 propose un ensemble fixe de cinq axes par défaut, identique pour tous les utilisateurs. Une évolution naturelle consiste à laisser chaque utilisateur configurer ses propres axes selon les spécificités de son secteur, via une interface de paramétrage dans le dashboard.

Versionnement et comparaison temporelle des rapports. La V1 produit chaque semaine un rapport indépendant. Ajouter une dimension temporelle qui permette de comparer la SWOT d'aujourd'hui à celle d'il y a un mois ou de visualiser l'évolution des facteurs PESTEL apporterait une valeur considérable pour les comités de direction.

Extension à d'autres types de veille. La V1 couvre strictement la veille concurrentielle. L'ajout de compétences dédiées à la veille réglementaire ou à la surveillance de la réputation en ligne, sans toucher au pipeline existant, ouvrirait le produit à de nouveaux cas d'usage.

Multi-tenant et passage à l'échelle. La V1 tourne sur une instance OpenClaw mono-VPS, adaptée à quelques dizaines d'utilisateurs au plus. Au-delà, il faudra penser une architecture multi-tenant horizontale avec répartition de charge, isolation réseau renforcée et migration de la queue interne vers un broker dédié.

Modèle freemium et internationalisation. La V1 est livrée gratuitement et uniquement en français. Un modèle freemium où la version payante mobilise un LLM haut de gamme (Claude Opus 4.7) pour des analyses plus fines, combiné à une interface disponible en anglais et en arabe, ouvrirait le marché au-delà du Maroc, vers le Maghreb francophone et le monde arabe.

Chapitre 8

Bibliographie et références

Ce chapitre regroupe sur une mise en deux colonnes les références mobilisées dans le projet RADAR. Les sources sont organisées par catégorie thématique pour faciliter la consultation. Chaque entrée comprend le nom de la ressource suivi de son lien direct, ce qui permet à tout lecteur de retrouver instantanément la documentation officielle ou le texte original.

Cours et supports académiques

Module M244 Veille Technologique : Pr
Younes Wadiai, ENSA Tétouan, 2025-2026
ENSA Tétouan :

<https://ensate.uae.ac.ma/>

Université Abdelmalek Essaadi :

<https://www.uae.ac.ma/>

Ouvrages et articles sur la veille stratégique

Lesca H. « La veille stratégique : concepts, méthodes, mise en oeuvre » : ASTI Éditions, 2003

Ansoff I. « Managing Strategic Surprise by Response to Weak Signals » : California Management Review, 1975 :

<https://doi.org/10.2307/41164635>

Blakeslee S. « The CRAAP Test » : LOEX Quarterly, 2004 : <https://commons.emich.edu/loexquarterly/vol31/iss3/4/>

Pateyron J.P. « La veille stratégique » : Economica

Bondu J. « Intelligence économique et veille stratégique » : Inter-Ligere

SWOT Analysis (origines) :

<https://www.businessballs.com/strategy-innovation/swot-analysis/>

PESTEL Analysis :

<https://www.cipd.org/en/knowledge/factsheets/pestle-analysis-factsheet/>
Larivet S. « Intelligence économique : enquête dans 100 PME » : L'Harmattan

Modèles de langage et frameworks d'agents

DeepSeek API Documentation :

<https://api-docs.deepseek.com/>

OpenClaw (dépôt officiel) :

<https://github.com/openclaw/openclaw>

OpenClaw documentation :

<https://docs.openclaw.ai/>

Anthropic Claude API :

<https://docs.anthropic.com/>

Anthropic « Building effective agents » :

<https://www.anthropic.com/engineering/building-effective-agents>

OpenAI Chat Completions API :

<https://platform.openai.com/docs/api-reference/chat>

LangChain :

<https://python.langchain.com/docs/>

AutoGPT : [https://github.com/](https://github.com/Significant-Gravitas/AutoGPT)

[Significant-Gravitas/AutoGPT](https://github.com/Significant-Gravitas/AutoGPT)

Recherche web et collecte de données

Tavily (AI-native search) :

<https://tavily.com/>

Tavily API Documentation :
<https://docs.tavily.com/>

DuckDuckGo Instant Answer API :
<https://duckduckgo.com/api>

Application web et persistance

Next.js 16 : <https://nextjs.org/docs>

React 19 : <https://react.dev/>

Tailwind CSS 4 :
<https://tailwindcss.com/docs>

Better-Auth :
<https://www.better-auth.com/>

Prisma 6 : <https://www.prisma.io/docs>

PostgreSQL 17 :
<https://www.postgresql.org/docs/17/>
Node.js 24 : <https://nodejs.org/en/docs/>

Infrastructure et déploiement

Docker Engine :
<https://docs.docker.com/engine/>

Docker Compose :
<https://docs.docker.com/compose/>

Hetzner Cloud :
<https://docs.hetzner.com/cloud/>

Let's Encrypt :
<https://letsencrypt.org/docs/>

Nginx : <https://nginx.org/en/docs/>

Outils de développement et qualité

Git : <https://git-scm.com/doc>

GitHub : <https://docs.github.com/>

TypeScript :
<https://www.typescriptlang.org/docs/>

pnpm : <https://pnpm.io/>

Turborepo : <https://turborepo.com/docs>

ESLint : <https://eslint.org/docs/>

Visual Studio Code :
<https://code.visualstudio.com/docs>

Outils de veille concurrentielle (benchmark)

Similarweb : <https://www.similarweb.com/>

SEMrush : <https://www.semrush.com/>

Crayon : <https://www.crayon.co/>

Mention : <https://mention.com/>

Brandwatch :
<https://www.brandwatch.com/>

Talkwalker : <https://www.talkwalker.com/>

Google Alerts :
<https://www.google.com/alerts>

Feedly : <https://feedly.com/>